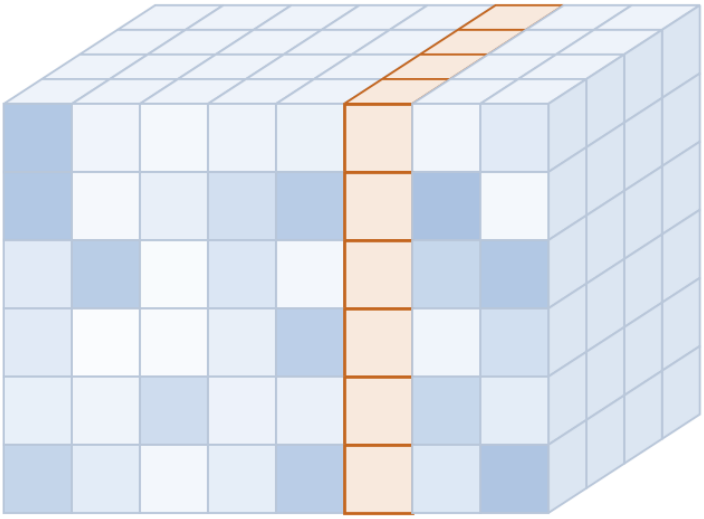
FULL-STACK DATA SCIENCE WITH PYTHON

An illustrated companion to the course notebooks



MODULE 02

NumPy & Numerical Computing

Thinking in whole arrays, and knowing what they cost

How to read this book

This book walks through every idea in the Module 02 notebooks, but it is written to be *read*, on a train or with a coffee, rather than executed. The notebooks remain the place where you run code, break it and fix it; the book is where you build the mental pictures that make the code obvious.

Each chapter mirrors one notebook and opens with the notebook's file name, so you can always jump between the two. Code in the book is trimmed to the lines that carry the idea, and every figure was generated from code in this repository, using the same colour palette as the notebooks.

Five kinds of boxes appear along the way. They always mean the same thing:

> **NOTE**
> Extra context. Worth knowing, but you can move on without it.

> **TIP**
> A habit or shortcut professionals actually use.

> **WATCH OUT**
> A mistake that produces silently wrong numbers, not an error.

> **FROM THE HISTORY OF DATA SCIENCE**
> A true episode from the history of science and industry that makes the idea stick. Sources are listed at the back.

> **KEY TAKEAWAYS**
> The handful of ideas to carry out of the chapter.

Contents

1 Why NumPy Exists . . . 6

1.1 The problem with a list of numbers . . . 6

1.2 Measuring the speed difference . . . 8

1.3 What "vectorised" really means . . . 9

1.4 Universal functions . . . 10

1.5 How the advantage scales . . . 10

1.6 When NumPy is not the answer . . . 10

1.7 Try it yourself . . . 12

2 The ndarray, Up Close . . . 13

2.1 Creating arrays . . . 13

2.2 The four attributes to check first . . . 14

2.3 Indexing and slicing . . . 15

2.4 Views versus copies: read this twice . . . 16

2.5 Reshaping . . . 17

2.6 Combining and splitting . . . 18

2.7 A practical checklist . . . 18

2.8 Try it yourself . . . 19

3 Numbers With Limits . . . 20

3.1 The dtype zoo . . . 20

3.2 Silent integer overflow . . . 21

3.3 Type promotion . . . 23

3.4 float32 versus float64 . . . 23

3.5 Epsilon and catastrophic cancellation . . . 25

3.6 NaN and inf . . . 25

3.7 Cutting memory with correct downcasting . . . 26

3.8 datetime64 and timedelta64 . . . 26

3.9 Try it yourself . . . 27

4 Broadcasting . . . 28

4.1 The two rules . . . 28

4.2 Reading the error message . . . 30

4.3 Say what you mean: np.newaxis . . . 30

4.4 The silent bug . . . 31
4.5 Real applications . . . 32
4.6 The memory trap . . . 33
4.7 Try it yourself . . . 34

5 Summarise, Select, Decide . . . 35

5.1 axis=: the only rule you need . . . 35
5.2 Boolean masks . . . 37
5.3 Decisions without branching . . . 38
5.4 Fancy indexing . . . 39
5.5 Counting and grouping . . . 39
5.6 Putting it together: a vectorised quality report . . . 40
5.7 Try it yourself . . . 41

6 Linear Algebra Without Tears . . . 43

6.1 Vectors . . . 43
6.2 Matrix multiplication . . . 44
6.3 Solving linear systems . . . 45
6.4 Least squares: linear regression from first principles . . . 46
6.5 Eigenvalues and eigenvectors . . . 48
6.6 Singular Value Decomposition . . . 49
6.7 Quick reference . . . 50
6.8 Try it yourself . . . 50

7 Dice You Can Replay . . . 52

7.1 The Generator API . . . 52
7.2 The distributions you'll use . . . 54
7.3 Sampling . . . 55
7.4 Monte Carlo . . . 55
7.5 The bootstrap . . . 56
7.6 Try it yourself . . . 58

8 Under the Hood . . . 59

8.1 Strides: the trick behind everything . . . 59
8.2 Temporaries: the hidden allocations . . . 61
8.3 Measuring memory honestly . . . 62
8.4 Arrays larger than RAM: np.memmap . . . 62
8.5 Choosing the right tool . . . 62

8.6 A performance checklist . . . 63
8.7 Try it yourself . . . 64

9 Project: Two Algorithms From Scratch . . . 65

9.1 Part 1: linear regression . . . 66
9.2 Part 2: k-means clustering . . . 68
9.3 Verification . . . 70
9.4 Your turn: extensions . . . 70
9.5 What you built . . . 70

Before You Move On . . . 72

Self-assessment . . . 72
The three traps this module exists to prevent . . . 72
Where the exercises and solutions live . . . 73
Further reading . . . 73

Sources for the Stories . . . 74

Chapter 1 . . . 74
Chapter 2 . . . 74
Chapter 3 . . . 74
Chapter 4 . . . 74
Chapter 5 . . . 75
Chapter 6 . . . 75
Chapter 7 . . . 75
Chapter 8 . . . 75
Chapter 9 . . . 76

CHAPTER 01

01

Why NumPy Exists

Memory layout, vectorisation, and the honest limits of arrays

Notebook 01_why_numpy_exists.ipynb · About 60 minutes

IN THIS CHAPTER

- Why a Python list of numbers is slow: the answer is **memory layout**, not just interpreter speed.
- What "vectorised" really means underneath.
- How to measure the difference yourself instead of taking it on faith.
- Universal functions (ufuncs), the interface every NumPy operation shares.
- When NumPy is *not* the right answer.

Every numerical library in this course (pandas, scikit-learn, statsmodels, SciPy, PyTorch) is a layer on top of one object: the NumPy array. This module is where that layer stops being magic. We start with the most basic question of all: why does it exist? Python already has lists.

1.1 The problem with a list of numbers

A Python `list` is a wonderful general-purpose container. It holds anything, in any mix, and grows on demand. That flexibility has a cost, and the cost is structural.

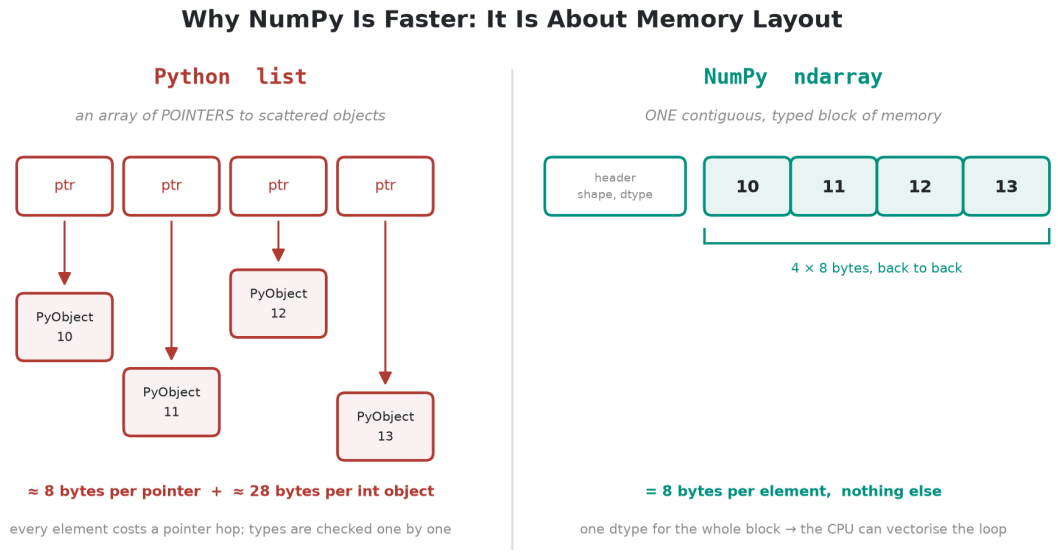


Figure 1.1 A list stores pointers to scattered, fully fledged Python objects. An ndarray stores the raw numbers themselves, back to back, all of one type.

A list doesn't store your numbers. It stores **pointers** to full Python objects living elsewhere in memory. Adding two lists element by element therefore means, for *every single element*:

1. follow a pointer to object A,
2. follow a pointer to object B,
3. check both types at runtime,
4. dispatch to the right `__add__`,
5. allocate a brand-new Python object for the result,
6. store a pointer to it.

An ndarray stores the raw numbers, contiguously, all of one type. Adding two arrays runs a tight loop in compiled C with none of steps 1 to 4.

Think of a library. A list is a card catalogue: each card tells you which shelf, in which room, holds the book you want, and you walk there every time. An ndarray is a single shelf with the books already lined up in order. Reading every book is a stroll along one shelf.

Seeing the overhead

```
sys.getsizeof(10)     # 28 bytes for one Python int object
np.int64(10).itemsize # 8 bytes for the same number inside an array
```

> **NOTE**
> Those extra bytes aren't waste. They hold the reference count, the type pointer and the arbitrary-precision digits that make Python integers unbounded (Module 01, Chapter 2). You're paying for real features. In numerical work you rarely need them, and NumPy lets you decline.

`sys.getsizeof` lies about containers

Module 01 flagged this: `getsizeof` is **shallow**. For a list it counts the array of pointers and nothing they point to. The honest figure adds up every referenced object too:

```
py_list = list(range(1_000_000))
np_array = np.arange(1_000_000, dtype=np.int64)

shallow = sys.getsizeof(py_list)                            # ~7.6 MiB
true_size = shallow + sum(sys.getsizeof(x) for x in py_list) # ~34 MiB
np_array.nbytes                                             # ~7.6 MiB, exactly
```

For an array there's nothing else to count: the data *is* the buffer, so `nbytes == size × itemsize`, exactly, always.

1.2 Measuring the speed difference

Let's settle Module 01's claim with a measurement rather than an assertion: four ways to sum a million integers.

```
def time_it(fn, *args, repeats: int = 3) -> float:
    """Best of `repeats` timings, in seconds."""
    best = float("inf")
    for _ in range(repeats):
        start = time.perf_counter()
        fn(*args)
        best = min(best, time.perf_counter() - start)
    return best
```

Why the *minimum* and not the average? Timing noise is one-sided: background processes can only ever make a run slower, never faster. The fastest observed run is the closest estimate of the true cost.

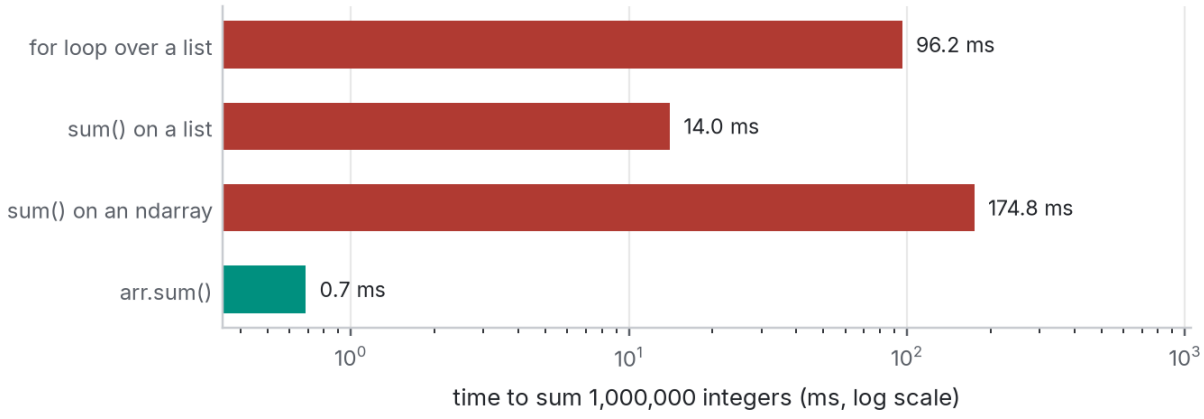


Figure 1.2 Summing a million integers four ways, measured on the machine that built this book. Note the logarithmic scale.

Two results are worth pausing on.

`sum()` on an ndarray is the slowest option of all. Python's built-in `sum` walks the array one element at a time, and NumPy has to box each raw value back into a Python object to hand it over. You get all of the list's overhead *plus* the boxing.

`arr.sum()` **is dramatically faster**, because the whole loop runs in compiled code over one contiguous buffer: no boxing, no type checks, no pointer chasing.

> WATCH OUT
> **Never apply a Python built-in to an array.** `sum(arr)`, `max(arr)` and `min(arr)` are slower than even the list versions. Use `arr.sum()`, `arr.max()`, `np.min(arr)`.

1.3 What "vectorised" really means

The word gets thrown around loosely. Precisely, it means: **the loop moves out of the Python interpreter and into compiled code.** Three consequences follow.

1. Operations apply to the whole array at once.

```
a = np.array([1, 2, 3, 4]); b = np.array([10, 20, 30, 40])
a + b        # [11 22 33 44]
a * b        # [10 40 90 160]  element-wise, NOT matrix multiplication
a > 2        # [False False  True  True]  comparisons vectorise too
```

2. There's no per-element type check. The dtype is fixed for the whole array, so the C loop checks it once.

3. Contiguity lets the CPU do several numbers per instruction. Modern processors have SIMD registers that add four or eight `float64`s in one go, but only when the numbers sit side by side in memory. Summing the same five million values through a *strided view* (every second element of a bigger array) is noticeably slower than summing them from a contiguous copy.

> TIP
> That's why `np.ascontiguousarray` exists, and why Chapter 8 spends time on strides. Slicing, transposing and reshaping can leave an array non-contiguous, and the cost is real.

> FROM THE HISTORY OF DATA SCIENCE
> ### A notation that became a way of thinking
> The idea of operating on whole arrays at once is older than NumPy, older than Python, older than most programming languages still in use. In 1962 Kenneth Iverson, then at Harvard and IBM, published *A Programming Language*, a mathematical notation in which `A + B` simply meant "add these two arrays, element by element". It became the language **APL**, beloved by actuaries and engineers for its terse power, and Iverson received the Turing Award in 1979 for a lecture titled *Notation as a Tool of Thought*. Every time you write `prices * quantities` instead of a loop, you're thinking in Iverson's notation.

1.4 Universal functions

A **ufunc** is a function that works element by element on arrays, with broadcasting, type promotion and an optional output buffer all handled for you. Nearly everything in NumPy is one:

```
values = np.array([1.0, 4.0, 9.0, 16.0])
np.sqrt(values)                      # [1. 2. 3. 4.]
np.maximum([1, 5, 3], [4, 2, 6])     # [4 5 6]  element-wise pairwise max
isinstance(np.sqrt, np.ufunc)        # True
```

Why `math.sqrt` is no substitute

The `math` module works on one scalar at a time. Applying it to an array means looping in Python, which is exactly what we're trying to avoid. On a million values, `np.sqrt(arr)` beats `[math.sqrt(v) for v in arr]` by a large factor.

`out=`: skip the allocation

Every ufunc can write into an array you already have instead of allocating a new one. On large arrays inside a loop, this matters:

```
buffer = np.empty(5_000_000)
np.sqrt(x, out=buffer)       # writes in place, allocates nothing
```

1.5 How the advantage scales

The gap isn't a fixed multiplier. It **widens with array size**, because Python's per-element overhead is constant while NumPy spreads its own fixed per-call cost over more and more data.

> **NOTE**
> At very small sizes the advantage shrinks, and it can even invert. NumPy pays a fixed cost per *call* (parsing arguments, resolving dtypes, allocating the result) that a plain Python operation avoids. For a handful of values, plain Python can win. Don't "NumPy-ify" a three-element list.

1.6 When NumPy is not the answer

Knowing a tool includes knowing its limits:

Situation	Why NumPy struggles	Reach for
Mixed columns (strings + numbers + dates)	One dtype per array	**pandas** (Module 03)
Genuinely variable-length records	Rectangular arrays only	lists of dicts, or Arrow
Data larger than RAM	Everything lives in memory	**Dask / Spark** (Module 20)
Heavy text processing	No string vectorisation worth the name	Python `str`, pandas `.str`

Situation	Why NumPy struggles	Reach for
An algorithm that can't avoid a loop	Python loop overhead returns	**Numba**, **Cython**, or C
Fewer than ~100 elements	Per-call overhead dominates	plain Python

NumPy *can* hold mixed data, but watch what happens to the dtype:

```
mixed = np.array([1, "two", 3.0])
mixed.dtype           # <U32: everything became a string
mixed[0] + mixed[0]   # '11'   ← string concatenation!
```

That silently destroys your numbers. It's exactly the problem pandas solves.

FROM THE HISTORY OF DATA SCIENCE

From three projects to one, and on to the stars

NumPy wasn't designed in one go. In 1995 Jim Hugunin, then a graduate student, wrote **Numeric**, the first serious array package for Python. In the early 2000s a team at the Space Telescope Science Institute built **Numarray** to handle very large astronomical images, and for a while the community was split between the two. In 2005 Travis Oliphant merged the best of both into a single package, and **NumPy 1.0** shipped in 2006. Underneath, it leaned on decades-old Fortran libraries, BLAS (1979) and LAPACK (1992), for its heaviest linear algebra.

The result became the foundation of scientific Python. When the LIGO observatories announced the first detection of gravitational waves in 2016, and when the Event Horizon Telescope released the first image of a black hole in 2019, NumPy was in the software stack. Both feats are described in the 2020 *Nature* paper on NumPy written by its maintainers.

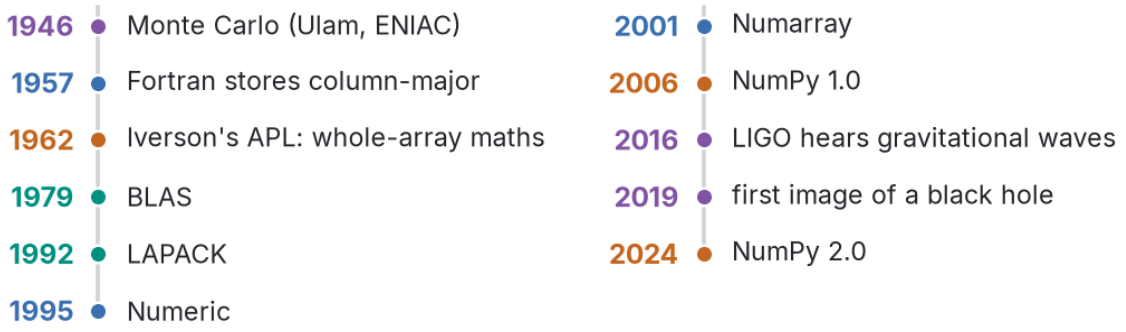


Figure 1.3 The milestones in this book's stories, from early Monte Carlo to NumPy 2.0.

1.7 Try it yourself

Exercises 1.1 to 1.5 in notebook 01; solutions in notebook 10.

1. **Measure it yourself.** Compute the element-wise product of two sequences of a million floats four ways: a `for` loop, a list comprehension, `zip` + comprehension, and `arr_a * arr_b`. Report each time and the speed-up.
2. **The honest memory question.** Write `true_list_size(lst)` counting the pointer array plus every referenced object, compare it with the equivalent `int64` array for 10, 1,000 and 100,000 elements, and plot the ratio.
3. **Find the break-even point.** For `x * 2 + 1`, find the size at which NumPy overtakes a list comprehension, searching from 1 to 10,000. Why isn't it 1?
4. **Spot the de-vectorisation.** Explain why each of these drops back to Python speed, and rewrite it: `sum(arr)`; `np.array([np.sqrt(v) for v in arr])`; `max(arr)`; `np.array(list(map(lambda x: x * 2, arr)))`.
5. **`out=` in a loop.** Run 100 rounds of `y = sqrt(x) + log1p(x)` over a million elements, once allocating fresh arrays and once reusing preallocated buffers. Measure both.

KEY TAKEAWAYS

- A list stores **pointers to objects**; an ndarray stores **raw values, back to back, all one dtype**. Every performance difference follows from that.
- `sys.getsizeof` **is shallow** and badly understates a list. `ndarray.nbytes` is exact.
- **Vectorised** means the loop runs in compiled C, not in the interpreter.
- **Never apply a Python built-in to an array**: `sum(arr)` is slower than the list version.
- Ufuncs accept `out=` to reuse a buffer instead of allocating.
- The advantage **grows with size** and can invert below ~100 elements.
- NumPy needs one dtype and a rectangular shape. When your data isn't like that, the answer is pandas, not a cleverer array.

CHAPTER 02

02

The ndarray, Up Close

Creation, shape, indexing, and the views-versus-copies rule

Notebook 02_ndarray_fundamentals.ipynb · About 90 minutes

IN THIS CHAPTER

- Every way to create an array, and which one to reach for.
- `shape`, `ndim`, `size`, `dtype`: the four attributes you'll check constantly.
- Indexing and slicing in N dimensions.
- **Views versus copies**, the rule that *reverses* what you learned about lists, and the biggest source of silent NumPy bugs.
- Reshaping, transposing, stacking and splitting.

Learning a new tool is a bit like moving into a new kitchen. Most things are where you'd expect, and then one drawer opens the other way. In NumPy, that drawer is slicing: the same `[1:3]` you used on lists all through Module 01 behaves completely differently on an array. We'll get there. First, the basics.

2.1 Creating arrays

From Python data

```
np.array([1, 2, 3, 4])              # dtype int64, shape (4,)
np.array([1.0, 2.0, 3.0])           # dtype float64
np.array([[1, 2, 3], [4, 5, 6]])    # nested lists → 2-D, shape (2, 3)
np.array([1, 2, 3], dtype=np.float64)   # explicit dtype: a good habit
```

Being explicit about the dtype documents intent and prevents the surprise of getting integers when you needed floats.

From scratch

```
np.zeros(3)           # [0. 0. 0.]
np.ones((2, 3))       # a 2×3 block of ones
np.full((2, 2), 7)    # filled with 7
np.eye(3)             # the identity matrix
np.empty(3)           # UNINITIALISED memory, see below
```

> **WATCH OUT**
> `np.empty` does **not** zero its memory. It hands you whatever bytes happened to be there already. It's marginally faster than `zeros` and is the right choice only when you're about to overwrite every element. Forget to fill it and you get garbage that looks plausible.

Ranges and grids

```
np.arange(0, 10, 2)      # [0 2 4 6 8]   like range(), but an array
np.linspace(0, 1, 5)     # [0. 0.25 0.5 0.75 1.]   N points, BOTH ends included
np.logspace(-3, 0, 4)    # [0.001 0.01 0.1 1.]   ideal for hyperparameter grids
```

> **TIP**
> **Prefer `linspace` to `arange` for floats.** `arange` steps by repeated addition, so floating-point error accumulates and the endpoint becomes unpredictable. `np.arange(0, 0.3, 0.1)` gives *three* values, `0.3` sneaking in at the end because `0.1 + 0.1 + 0.1` is slightly less than `0.3` in binary. `linspace` asks for a count and gets exact ends.

Random arrays

Always use the modern `Generator` API. The legacy `np.random.seed` / `np.random.rand` functions share one global state, which makes reproducibility fragile as soon as two pieces of code draw numbers (Chapter 7 shows exactly how).

```
rng = np.random.default_rng(seed=42)
rng.random(3)                  # uniform on [0, 1)
rng.normal(0, 1, 3)            # standard normal
rng.integers(0, 10, 5)         # integers in [0, 10)
rng.choice(["a", "b"], 4)
```

Same seed, same numbers, on every machine.

2.2 The four attributes to check first

When something goes wrong in NumPy code, the answer is almost always in one of these. Print them before you debug anything else:

```
arr = rng.normal(size=(3, 4, 2))
arr.shape      # (3, 4, 2)  size along each dimension
arr.ndim       # 3          number of dimensions
arr.size       # 24         total elements
arr.dtype      # float64    type of every element
arr.nbytes     # 192        size × itemsize
```

1-D is not 2-D

`(3,)`, `(3, 1)` and `(1, 3)` hold the same three numbers and behave completely differently under broadcasting (Chapter 4) and matrix multiplication (Chapter 6).

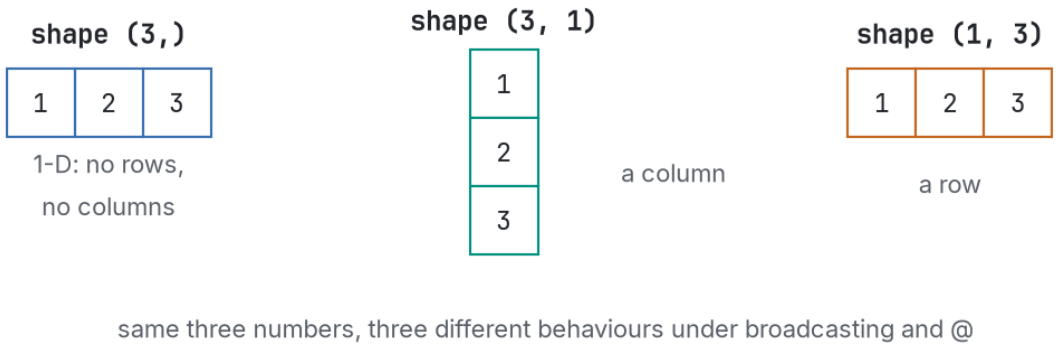


Figure 2.1 Three shapes, three behaviours. A 1-D array has no rows and no columns; a column and a row are both 2-D.

A taste of the trouble ahead: `np.array([1, 2, 3]) + np.array([[1], [2], [3]])` doesn't give three numbers. It gives a 3×3 matrix. Chapter 4 explains exactly why.

2.3 Indexing and slicing

```
arr = np.arange(1, 13).reshape(3, 4)
arr[0, 0]      # 1    one pair of brackets: [row, col]
arr[2, 3]      # 12
arr[-1, -1]    # 12   negative indexes work on every axis
```

`arr[1][2]` also works, but it's slower: it builds an intermediate row array first, then indexes that. Write `arr[1, 2]`.

Slicing works per axis, separated by commas:

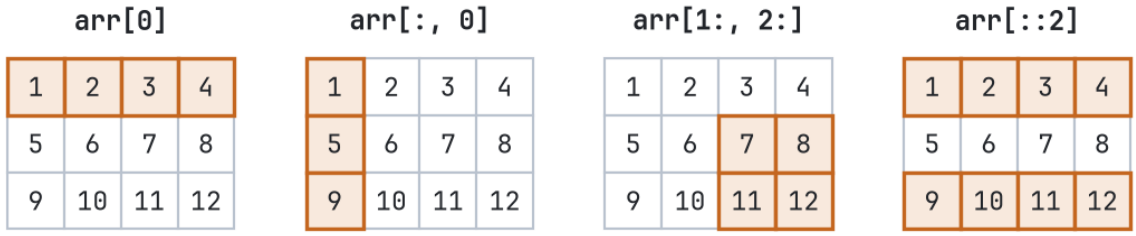


Figure 2.2 Four common selections on the same 3×4 array. Highlighted cells are the ones returned.

```
arr[0]         # first row
arr[:, 0]      # first column
arr[1:, 2:]    # bottom-right block
arr[::2]       # every other row
arr[:, ::-1]   # columns reversed
```

`...`: "all the remaining axes"

The Ellipsis saves typing when you only care about the first or last dimension: on a `(2, 3, 4)` array, `cube[0, ...]` is `cube[0, :, :]` and `cube[..., 0]` is `cube[:, :, 0]`.

2.4 Views versus copies: read this twice

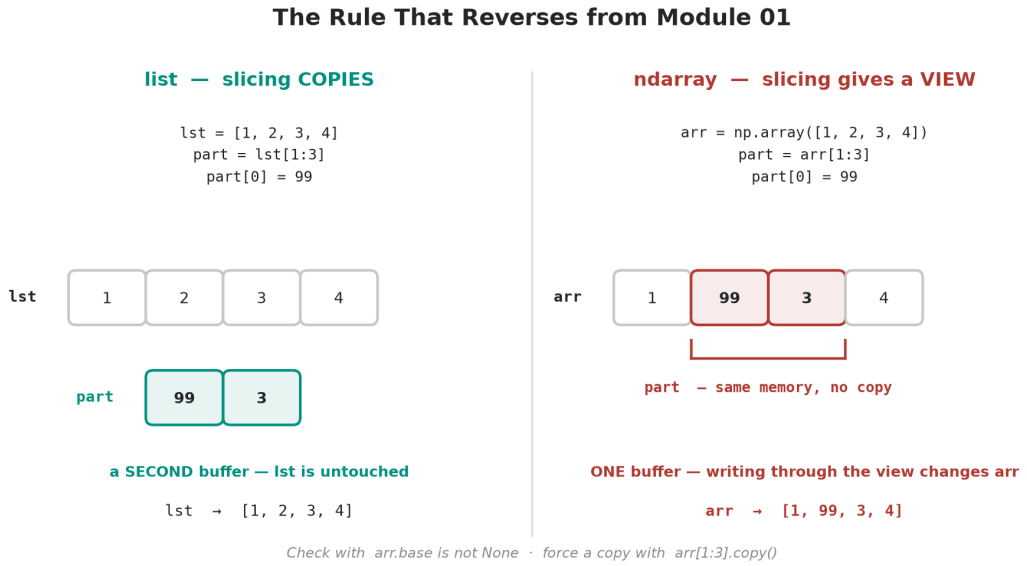


Figure 2.3 A slice of an array is a view onto the same memory. Writing through it changes the original.

In Module 01 you learned that slicing a list **copies.** In NumPy, **basic slicing returns a view**: a second window onto the *same* memory. Write through the view, and the original changes.

```
lst = [1, 2, 3, 4]
part = lst[1:3]; part[0] = 99
lst                     # [1, 2, 3, 4]   unchanged: a copy was made

arr = np.array([1, 2, 3, 4])
part = arr[1:3]; part[0] = 99
arr                     # [ 1 99  3  4]  CHANGED: part was a view
```

It's a deliberate design decision. It makes slicing free no matter how big the array is: no data is copied, only a new description of where to look. It is also the source of a whole category of silent bugs.

Think of a view as a window cut into a wall. Looking through a different window doesn't give you a different room. Paint the wall you see through it, and it's painted for everyone.

How to tell

```
original = np.arange(10)
view = original[2:5]
copy = original[2:5].copy()

view.base is original              # True: a view borrows its memory
copy.base                          # None: a copy owns its data
np.shares_memory(original, view)   # True
np.shares_memory(original, copy)   # False
```

What gives a view, what gives a copy

Operation	Result
`arr[1:5]`, `arr[:, 0]` (basic slicing)	**view**
`arr.reshape(...)`, `arr.ravel()`	**view** *when possible*
`arr.T`, `arr.transpose()`	**view**
`arr[[0, 2, 4]]` (fancy indexing)	**copy**
`arr[arr > 3]` (boolean mask)	**copy**
`arr.flatten()`, `arr.copy()`	**copy**, always
`arr + 1`, `np.sqrt(arr)`	**new array**

The bug this causes in practice

```
def normalise_first_column_broken(data: np.ndarray) -> np.ndarray:
    column = data[:, 0]          # a view
    column -= column.mean()      # in-place → writes into `data`
    return data

def normalise_first_column(data: np.ndarray) -> np.ndarray:
    out = data.copy()            # work on our own copy
    out[:, 0] = out[:, 0] - out[:, 0].mean()
    return out
```

The broken version silently rewrites the caller's array.

> **WATCH OUT**
> Notice *why* it bites: `column -= ...` is an **in-place** operation on a view. Had it been `column = column - column.mean()`, it would merely have rebound a local name and changed nothing at all. It's the `+=` versus `= ... +` distinction from Module 01's Chapter 2, with higher stakes.

2.5 Reshaping

```
flat = np.arange(12)
flat.reshape(3, 4)       # 3 rows, 4 columns
flat.reshape(2, -1)      # -1 means "work it out": (2, 6)
```

Use `-1` so the code survives a change in the number of rows.

The most common reshape in machine learning turns a 1-D array into a single-column 2-D array, because scikit-learn demands 2-D feature matrices. Three spellings, all views:

```
feature = np.array([1.0, 2.0, 3.0, 4.0])
feature.reshape(-1, 1).shape     # (4, 1)
feature[:, np.newaxis].shape     # (4, 1)
feature[:, None].shape           # (4, 1), the terse version
```

`ravel` versus `flatten`

Both give you a 1-D array. `ravel` returns a view when it can (cheap, but writes go through to the original); `flatten` always copies (safe).

Transposing

`m.T` swaps the axes and costs nothing: it only reinterprets the strides (Chapter 8). But the result is no longer C-contiguous, which can slow later operations.

2.6 Combining and splitting

```
a = np.array([[1, 2], [3, 4]]); b = np.array([[5, 6], [7, 8]])
np.vstack([a, b])                    # stack rows → (4, 2)
np.hstack([a, b])                    # stack columns → (2, 4)
np.concatenate([a, b], axis=0)       # the general form
np.stack([a, b]).shape               # (2, 2, 2): a NEW leading axis
```

`stack` differs from `concatenate`: it adds a new axis rather than extending an existing one. That's how you build a batch out of individual samples.

Splitting is the inverse. Splitting at explicit indices is how a train/validation/test split works before scikit-learn does it for you:

```
data = np.arange(12).reshape(6, 2)
train, validate, test = np.split(data, [4, 5])    # 4, 1 and 1 rows
```

2.7 A practical checklist

When an array misbehaves, run through the same questions every time. The notebook wraps them in a `describe(arr)` helper worth keeping in your back pocket: shape, ndim, dtype, nbytes, whether it's a view (`arr.base is not None`), contiguity (`arr.flags["C_CONTIGUOUS"]`), whether it's writeable, whether every value is finite, and its range.

> FROM THE HISTORY OF DATA SCIENCE
>
> **What's in a name?**
>
> "ndarray" stands for *N-dimensional array*, and the word "matrix" for a rectangular block of numbers is itself only about 170 years old. The English mathematician James Joseph Sylvester coined it in 1850, from the Latin for "womb": a matrix was the thing from which smaller determinants are *born*. A few years later his friend Arthur Cayley worked out the algebra of matrices: how to add them, multiply them and invert them. Every `reshape` and `@` you write rests on their Victorian groundwork.

2.8 Try it yourself

Exercises 2.1 to 2.5 in notebook 02; solutions in notebook 10.

1. **Predict the shapes.** For `a = np.arange(24).reshape(2, 3, 4)`, give the shapes of `a[0]`, `a[:, 1]`, `a[..., 2]`, `a[0, :, ::2]`, `a.T` and `a.reshape(4, -1)` without running anything.
2. **View or copy?** For `a = np.arange(20).reshape(4, 5)`, decide whether each shares memory with `a`, then check with `np.shares_memory`: `a[1:3]`, `a[:, [0, 1]]`, `a.T[0]`, `a[a % 2 == 0]`, `a.reshape(5, 4)`, `a.ravel()[::2]`.
3. **Fix the mutation bug.** A `scale_features(X, factor)` loops over columns doing `col = X[:, j]; col *= factor`. Show it corrupts the caller's array, rewrite it, and add an assertion that would have caught it.
4. **Build a checkerboard.** Make an 8×8 array of alternating 0s and 1s with no Python loop, then a second way. Hint: slicing with a step, or `np.indices`.
5. **Reshape for scikit-learn.** Turn `y = np.array([1.0, 2.0, 3.0])` into shape `(n, 1)` three ways and confirm all three share memory with `y`.

KEY TAKEAWAYS

- Check `shape`, `ndim`, `dtype`, `size` first when debugging. Most NumPy bugs are shape bugs.
- `(3,)`, `(3, 1)` and `(1, 3)` are three different things.
- Prefer `linspace` over `arange` for float ranges.
- Use `np.random.default_rng(seed)`, never the legacy global functions.
- **Basic slicing returns a view**, the opposite of list slicing. Fancy indexing and boolean masks return copies.
- Check with `.base` or `np.shares_memory`; force independence with `.copy()`.
- In-place operators (`-=`, `*=`) on a view write through to the original.
- `reshape(-1, 1)` makes the column vector scikit-learn wants.

CHAPTER 03

03

Numbers With Limits

Dtypes, silent overflow, and the precision you can actually trust

Notebook 03_dtypes_and_precision.ipynb · About 75 minutes

IN THIS CHAPTER

- The dtype zoo, and how to choose deliberately rather than by accident.
- **Silent integer overflow**, Module 01's warning paid in full.
- Type promotion, including what changed in NumPy 2.0.
- `float32` versus `float64`: the memory, speed and accuracy trade-off, quantified.
- Machine epsilon and catastrophic cancellation.
- `NaN` and `inf`: how they spread, and how to control the warnings.
- Cutting a dataset's memory dramatically with correct, *verified* downcasting.

Python's own integers are generous: they grow as big as you need. NumPy's are thrifty: each one gets a fixed number of bytes, decided in advance. That thrift is the source of NumPy's speed, and of its sharpest edge. This chapter is about knowing exactly where the edge is.

3.1 The dtype zoo

NumPy uses **fixed-width** types. Each has a hard minimum and maximum:

dtype	bytes	range
`int8`	1	−128 to 127
`int16`	2	−32,768 to 32,767
`int32`	4	about ±2.1 billion
`int64`	8	about $\pm 9.2 \times 10^{18}$
`uint8` … `uint64`	1 … 8	0 to twice the signed maximum, roughly

```
np.iinfo(np.int32).max      # 2147483647
np.finfo(np.float32).precision, np.finfo(np.float64).precision   # 6, 15
```

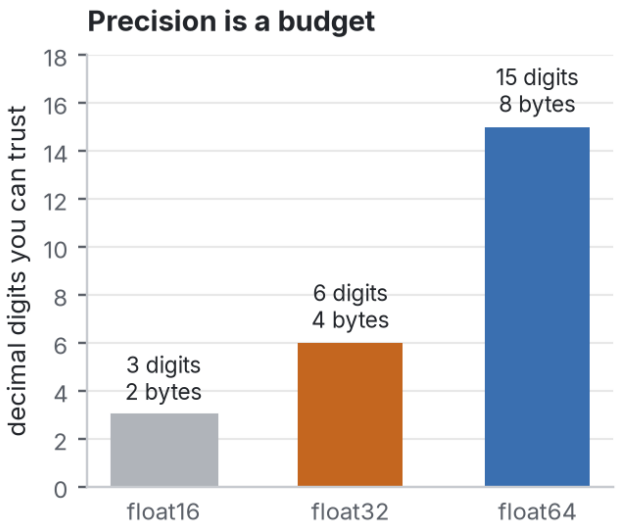


Figure 3.1 Precision is a budget: the number of decimal digits each float type can be trusted with.

`precision` is the number of **decimal digits you can trust.** `float32` gives about 7 (NumPy reports a conservative 6); `float64` about 15. That single number decides whether a computation is trustworthy, and it's worth memorising.

3.2 Silent integer overflow

A fixed-width integer is an odometer. Push it past its largest value and it rolls over to the smallest:

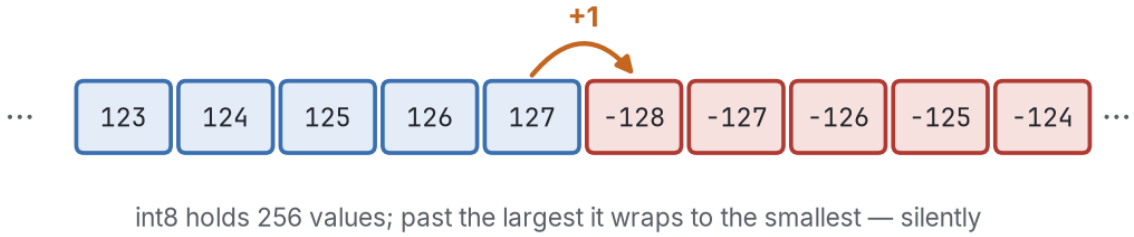


Figure 3.2 int8 holds 256 values. One step past 127 lands on −128, without a word.

```
arr = np.array([2**62], dtype=np.int64)
(arr * 2)[0]        # -9223372036854775808: wrapped to the most negative value
```

The dangerous part: arrays wrap without a warning

```
np.array([127], dtype=np.int8) + 1     # array([-128])   no warning at all
np.int8(127) + 1                       # -128, WITH a RuntimeWarning
```

The scalar path warns; the array path doesn't. That inconsistency makes the problem easy to miss in exploration and lethal in production.

WATCH OUT

Array arithmetic overflows silently. No exception, no warning, just a wrong number flowing into every downstream aggregate. In a revenue pipeline it shows up as a negative total; in an ID column it shows up as collisions. Neither is caught by a test that only checks for nulls.

FROM THE HISTORY OF DATA SCIENCE

Thirty-seven seconds

On 4 June 1996 the first Ariane 5 rocket lifted off from Kourou, French Guiana. About thirty-seven seconds later it veered off course and self-destructed. The inquiry traced the failure to a few lines of code reused from Ariane 4: a value related to the rocket's **horizontal velocity**, held as a 64-bit float, was converted into a **16-bit signed integer**. Ariane 5 was faster than Ariane 4 had ever been, the number no longer fitted, and the conversion raised an exception that shut down both inertial reference units in turn. The code had been correct for the old rocket. Nobody had re-checked that the *values* still fitted the *type*, exactly the check this chapter will teach you to automate.

Where it bites, and where it doesn't

The intuitive answer here is wrong, so let's get it exactly right. Store daily page views as `int32` to save memory: 400 days of 8 million views each, a true total of 3.2 billion, well past the `int32` ceiling.

```
daily_views = np.full(400, 8_000_000, dtype=np.int32)
daily_views.sum()                   # 3,200,000,000 ✔ (and its dtype is int64!)
daily_views.sum(dtype=np.int32)     # wrapped ✘
(daily_views * 400)[0]              # wrapped ✘
```

Reductions are safer than you'd expect; element-wise arithmetic is not. `sum`, `mean`, `prod` and `cumsum` promote their *accumulator* to the platform's default integer when the input is narrower, so `int32_array.sum()` is safe out of the box. Element-wise operations (`*`, `+`, `**`) get no such protection: the result keeps the input dtype and wraps in silence. **That's where the bug lives:** not in the aggregation you were worried about, but in the innocent `views * price` two lines earlier.

The defence for element-wise work is to widen *first*, on purpose:

```
views.astype(np.int64) * 2          # correct
```

Detecting overflow instead of hoping

`np.errstate` controls how NumPy reports numeric errors. Setting it to `"raise"` turns silence into an exception:

```
with np.errstate(over="raise"):
    np.multiply(values, 4, dtype=np.int32)   # FloatingPointError
```

3.3 Type promotion

When operands differ, NumPy promotes to a type that can hold both. NumPy 2.0 changed the rules materially (the change is called **NEP 50**), so examples written before 2023 may not match what you see.

Expression	Result dtype
`int32` array + `int64` array	`int64`
`int32` array + `float32` array	`float64`
`float32` array + `float64` array	`float64`
`int` array + `bool` array	`int64`
`float32` array + Python `float`	`float32`
`int32` array + Python `int`	`int32`

The last two rows are the NEP 50 change: **a Python scalar no longer upcasts an array.** `float32_array + 1.0` stays `float32`. Before 2.0 it became `float64`, silently doubling memory in deep-learning pipelines. "The array wins" is more predictable, and usually what you wanted.

You can ask what two types will produce without running anything: `np.promote_types(np.int64, np.float32)` is `float64`, because neither type can hold every value of the other.

Division always produces a float

```
a = np.array([7, 8, 9], dtype=np.int32)
a / 2       # float64: true division, and the memory just doubled
a // 2      # int32:   floor division keeps the integer type
```

3.4 `float32` versus `float64`

`float64` is NumPy's default and the right choice most of the time. `float32` halves the memory and often runs faster, at the cost of about eight decimal digits. Deep learning uses `float32` (and even `float16`) everywhere; statistics usually shouldn't.

Absorption

The starkest failure: add a small number to a big one, and the small one vanishes, because the result can't represent the difference.

```
np.float32(1e8) + np.float32(1) == np.float32(1e8)   # True: the 1 is GONE
np.spacing(np.float32(1e8))                          # 8.0
```

Near 10^8, consecutive `float32` values are 8 apart. There is simply no `float32` for 100,000,001.

Accumulation

Sum ten million copies of 0.1 and compare with the true million:

- `float64`: off by a few billionths.

- `float32` via `.sum()`: off by about 0.1. Far better than you might fear, because **NumPy's `.sum()` uses pairwise summation**, which makes the error grow like √n instead of n. You're benefiting from a good algorithm, not from `float32` being accurate.

Write the naive running total yourself and that protection disappears:

```
naive = np.float32(0.0)
for _ in range(1_000_000):
    naive = np.float32(naive + np.float32(0.1))
# about 1% off. From a loop that looks completely reasonable.
```

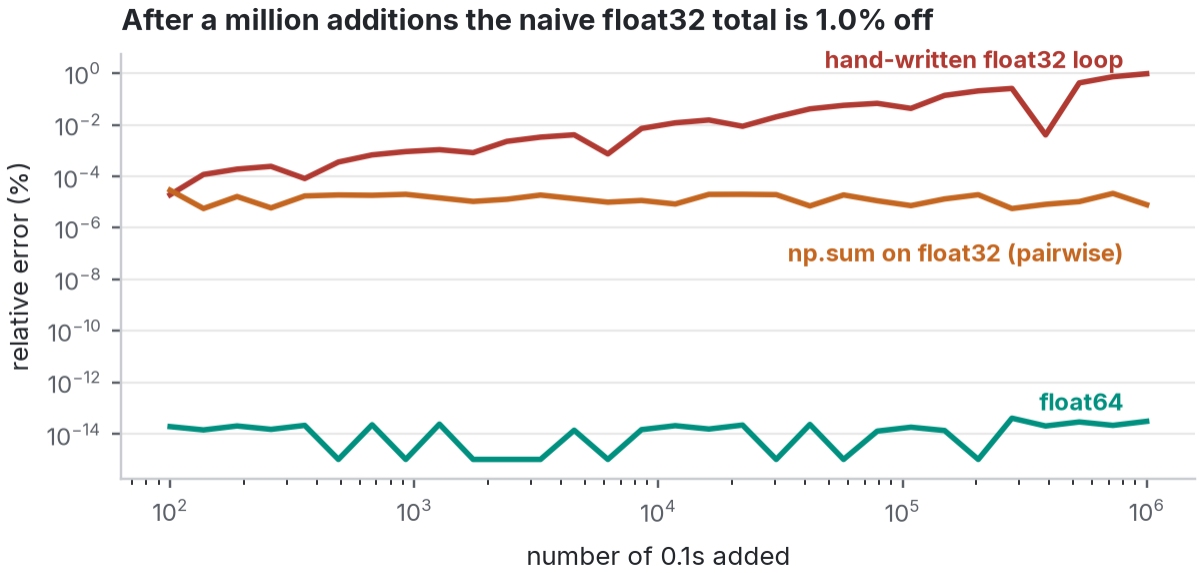


Figure 3.3 Measured: relative error of a running float32 total versus NumPy's pairwise sum and float64, as more 0.1s are added. Both axes are logarithmic.

When you must sum `float32` data, widen the accumulator: `values_32.sum(dtype=np.float64)`.

FROM THE HISTORY OF DATA SCIENCE

A third of a second, after a hundred hours

During the Gulf War, a Patriot missile battery at Dhahran, Saudi Arabia, had been running continuously for about 100 hours when, on 25 February 1991, it failed to intercept an incoming Scud missile. The Scud hit a US Army barracks, killing 28 soldiers. The US General Accounting Office (today's Government Accountability Office) traced the cause to the system's clock: time was counted in tenths of a second, and **0.1 has no exact binary representation.** Stored in a 24-bit register, each tick was off by a tiny amount, and after 100 hours of accumulation the error was about 0.34 seconds. At the speed of a Scud, that shifted the tracking window by more than half a kilometre. The arithmetic on this page, small error times huge repetition, is not academic.

3.5 Epsilon and catastrophic cancellation

Machine epsilon is the smallest number that still changes 1.0 when you add it. It sets the resolution of the whole type: about 1.2×10^{-7} for `float32` and 2.2×10^{-16} for `float64`. Add *half* of epsilon to 1.0 and it's lost entirely.

Catastrophic cancellation

Subtracting two nearly equal numbers wipes out precision: the leading digits cancel, and what's left is mostly rounding error. The classic victim is the textbook variance formula:

```
data = np.array([1e8 + 1, 1e8 + 2, 1e8 + 3, 1e8 + 4])
(data**2).mean() - data.mean()**2     # E[x²] - E[x]²: nonsense
((data - data.mean())**2).mean()      # two-pass formula: 1.25 ✔
np.var(data)                          # 1.25 ✔
```

The naive formula subtracts two numbers around 10^{16} to get an answer around 1. Sixteen significant digits cancel, and `float64` only has about fifteen, so the result is pure noise. That's why you call `np.var` instead of typing the formula: **the library implements the stable version.** The lesson generalises: for standard numerical operations, prefer the library function to the textbook expression.

> NOTE
> The rules that make all of this predictable, on every computer, come from the **IEEE 754** standard of 1985, largely shaped by the Berkeley mathematician William Kahan, often called "the father of floating point". Before it, different machines rounded differently and the same program could give different answers on different hardware. Kahan received the Turing Award in 1989 for this work.

3.6 `NaN` and `inf`

```
special = np.array([1.0, np.nan, np.inf, -np.inf, 0.0])
np.isnan(special)       # [F  T  F  F  F]
np.isinf(special)       # [F  F  T  T  F]
np.isfinite(special)    # [T  F  F  F  T]  ← usually the one you want
```

NaN spreads through everything

One NaN poisons a sum or a mean. The `nan*` family skips missing values instead: `np.nansum`, `np.nanmean`, `np.nanstd`, `np.nanmax`.

> WATCH OUT
> The `nan*` functions skip silently, which is convenient and sometimes wrong. `np.nanmean([1, nan, nan, nan])` is `1.0`, computed from a single observation, and it won't tell you. **Always report the count alongside:** `np.isfinite(x).sum()`.

Controlling the warnings

Division by zero warns by default and produces `inf`, `-inf` or `nan`. While debugging, make it raise instead; it's far easier than hunting a NaN backwards through ten transformations. For deliberate division with a guard, `np.divide(..., where=...)` is the clean pattern:

```
with np.errstate(divide="raise"):
    1.0 / a                                    # FloatingPointError

safe = np.divide(1.0, a, out=np.full_like(a, np.nan), where=(a != 0))
```

3.7 Cutting memory with correct downcasting

Choosing the narrowest **safe** dtype is one of the highest-leverage optimisations in data work, but "safe" must be **verified**, not guessed. The notebook's `safe_downcast` checks each integer column's actual minimum and maximum against each candidate type's limits, and demotes a float column to `float32` only if the round trip is lossless for *this* data:

```
if np.issubdtype(arr.dtype, np.integer):
    low, high = arr.min(), arr.max()
    candidates = ([np.uint8, np.uint16, np.uint32] if low >= 0
                  else [np.int8, np.int16, np.int32])
    for candidate in candidates:
        info = np.iinfo(candidate)
        if low >= info.min and high <= info.max:
            return arr.astype(candidate)
```

On a million-row dataset (user IDs, ages, session counts, scores, a boolean), this removes the large majority of the memory. The notebook then **checks every column survived** before celebrating. A downcast that loses data isn't an optimisation.

> WATCH OUT
>
> **Downcast IDs with care.** An `int32` holds up to about 2.1 billion: fine for a user ID today, a silent collision the day the table grows. When a column is a *key* rather than a *measurement*, keep it wide, or store it as a string.

3.8 `datetime64` and `timedelta64`

```
dates = np.array(["2024-01-15", "2024-06-30", "2024-12-31"], dtype="datetime64[D]")
np.diff(dates)                                 # gaps in days
dates + np.timedelta64(7, "D")                 # one week later
(dates - np.datetime64("2024-01-01")).astype(int)   # days since New Year
```

The `[D]` is the resolution: day. Change it with `.astype("datetime64[h]")`.

> NOTE
>
> `datetime64` has no time zone; it represents a naive instant. For anything involving zones or calendars, pandas' `Timestamp` (Module 03) is the right tool: it wraps `datetime64` and adds the missing semantics.

3.9 Try it yourself

Exercises 3.1 to 3.5 in notebook 03; solutions in notebook 10.

1. **Predict the dtype** of: `int8 + int16` arrays; `int32 / 2`; `float32 array + 1.0`; `float32 array + np.float64(1.0)`; `bool + int` arrays; `np.arange(3) ** 2`.
2. **Trigger, then prevent, an overflow.** Build an `int16` array whose sum overflows; show the wrong answer, then produce the right one two different ways.
3. **When is float32 not enough?** Find the size at which summing `float32` 0.1s drifts more than 0.1% from the truth, and plot relative error against size for both float types.
4. **Build a memory report.** `memory_report(arrays)` prints each column's dtype, size, narrowest safe dtype and potential saving, and refuses to suggest downcasting any column whose name ends in `_id`.
5. **Cancellation in the wild.** Compute the standard error of a proportion, `sqrt(p*(1-p)/n)`, for `p = 1 - 1e-17` and `n = 1000`, directly and via a rearranged form. Explain the difference.

KEY TAKEAWAYS

- NumPy dtypes are **fixed-width.** Integers wrap; they don't grow.
- **Array overflow is silent.** Only the scalar path warns.
- **Reductions promote their accumulator; element-wise arithmetic doesn't.** The bug is almost always in the multiplication, not the sum.
- `np.errstate(over="raise")` turns silence into an exception while you debug.
- Since NumPy 2.0 (**NEP 50**), a Python scalar no longer upcasts an array.
- `float32` **≈ 7 digits,** `float64` **≈ 15.** Near 10^8, `float32` can't even represent +1.
- `.sum()` uses **pairwise summation**; your hand-written loop doesn't.
- Never write `E[x²] - E[x]²`. Use `np.var`.
- `nan*` functions skip NaN: **always report the count too.**
- **Downcasting must be verified**, and ID columns should stay wide.

CHAPTER 04

04 Broadcasting

Two rules for combining arrays of different shapes

Notebook 04_broadcasting.ipynb · About 75 minutes

IN THIS CHAPTER

- The two rules that completely decide whether two shapes are compatible.
- How to read, and fix, a broadcasting error.
- `np.newaxis` and `reshape` to make your intent explicit.
- The silent bug: shapes that broadcast happily into the wrong answer.
- Real applications: standardising features, pairwise distances, one-hot encoding.
- The memory trap, and how algebra, `einsum` and chunking avoid it.

Imagine handing out name badges at a conference. You don't print a separate copy of the event logo for each badge; you print the logo once and stamp it onto every one. Broadcasting is NumPy's version of that stamp: a smaller array is *virtually* stretched to match a bigger one, without ever being copied.

4.1 The two rules

Broadcasting lets NumPy combine arrays of different shapes without you writing a loop or materialising a copy. There are exactly two rules.

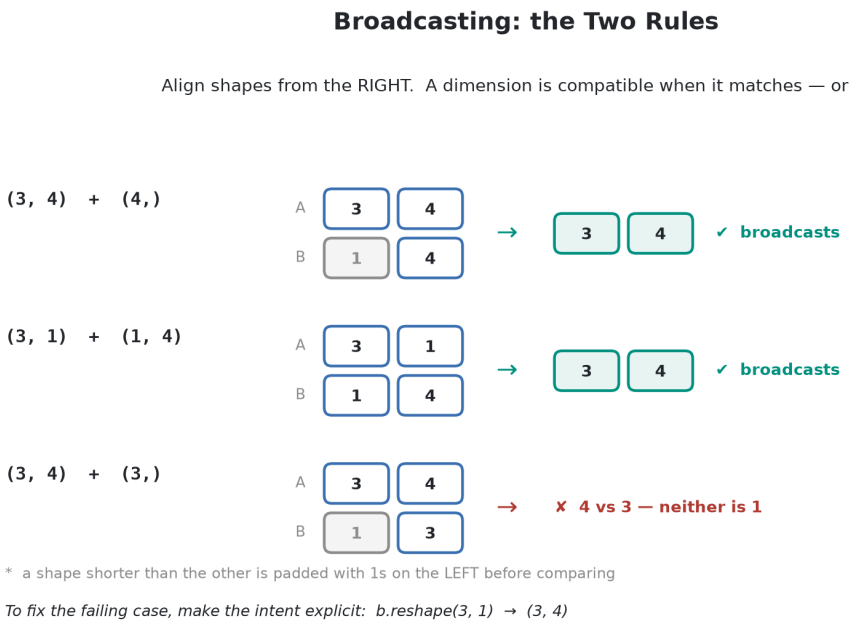


Figure 4.1 The two broadcasting rules: align shapes from the right, then compare each pair of dimensions.

1. **Align the shapes from the right.** If one is shorter, pad it on the **left** with 1s.
2. **Two dimensions are compatible if they're equal, or if one of them is 1.** A dimension of size 1 is *stretched* to match the other.

If any pair fails rule 2, the operation raises. That's the whole thing.

```
arr = np.array([1, 2, 3, 4])
arr + 10        # a scalar is shape (), padded to (1,), stretched to (4,)

matrix = np.arange(12).reshape(3, 4)
row = np.array([10, 20, 30, 40])
matrix + row    # (4,) → (1, 4) → stretched down all 3 rows
```

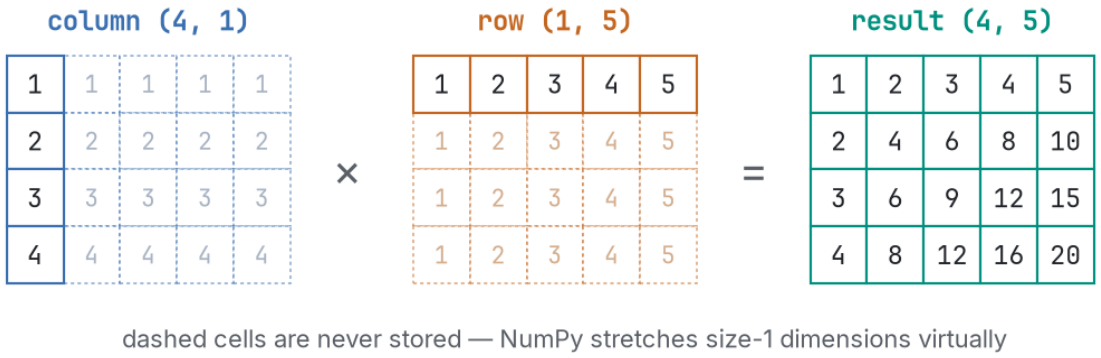


Figure 4.2 A column of shape (4, 1) times a row of shape (1, 5). The dashed cells are never stored; NumPy reads the same values again.

Checking compatibility without computing anything

`np.broadcast_shapes` answers "will these combine, and into what?". It allocates nothing, so it's perfect for debugging:

Shape A	Shape B	Result
`(3, 4)`	`(4,)`	`(3, 4)`
`(3, 4)`	`(3, 1)`	`(3, 4)`
`(3, 1)`	`(1, 4)`	`(3, 4)`
`(2, 3, 4)`	`(3, 4)`	`(2, 3, 4)`
`(5, 1, 3)`	`(4, 3)`	`(5, 4, 3)`
`(3, 4)`	`(3,)`	**incompatible**
`(2, 3)`	`(3, 2)`	**incompatible**

4.2 Reading the error message

```
np.arange(12).reshape(3, 4) + np.array([1, 2, 3])
# ValueError: operands could not be broadcast together with shapes (3,4) (3,)
```

Decode it by aligning from the right, just as NumPy does:

```
(3, 4)
   (3)   →  padded to (1, 3)
_________

 4 vs 3  →  neither is 1  →  fail
```

The intent was almost certainly "add one value per **row**", which means `b` needs to be a **column**: `a + b.reshape(-1, 1)` works, and each row gets its own value.

4.3 Say what you mean: `np.newaxis`

`np.newaxis` (an alias for `None`) inserts a dimension of size 1. It's the clearest way to say "treat this as a row" or "treat this as a column":

```
v = np.array([1, 2, 3])
v[np.newaxis, :].shape    # (1, 3)  a ROW
v[:, np.newaxis].shape    # (3, 1)  a COLUMN
v[:, None] * v[None, :]   # the outer product, (3, 3); same as np.outer(v, v)
```

4.4 The silent bug

The dangerous case isn't the one that raises. It's the one that **broadcasts successfully into a shape you never intended.**

```
predictions = np.array([1.0, 2.0, 3.0, 4.0])          # (4,)
actuals = np.array([[1.1], [2.1], [2.9], [4.2]])      # (4, 1): a column!

errors = predictions - actuals                         # (4, 4)  ← surprise
```

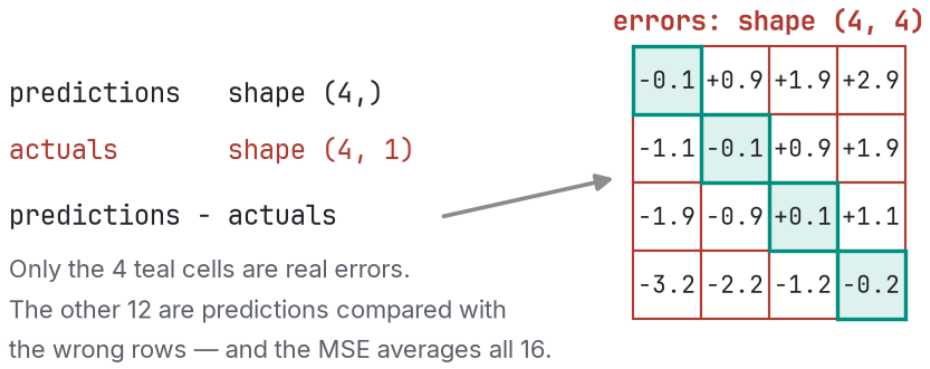


Figure 4.3 Every prediction subtracted from every actual. Only the diagonal is meaningful, yet the mean squared error averages all sixteen cells.

The "MSE" computed from that matrix is several times too large, and nothing raised.

> **WATCH OUT**
> This is the most common broadcasting bug in machine-learning code. It usually arrives as an `(n, 1)` column from `reshape(-1, 1)`, or from a DataFrame column meeting a 1-D prediction array. Nothing raises; the loss is simply wrong.

The defence: assert the shape you expect. A metric function that flattens both inputs and checks they match makes the bug impossible:

```
def mean_squared_error(y_true, y_pred) -> float:
    y_true = np.asarray(y_true).ravel()
    y_pred = np.asarray(y_pred).ravel()
    if y_true.shape != y_pred.shape:
        raise ValueError(f"shape mismatch: {y_true.shape} vs {y_pred.shape}")
    return float(((y_true - y_pred) ** 2).mean())
```

4.5 Real applications

Standardising features: what `StandardScaler` does

```
X = rng.normal(loc=[0, 100, -50, 1000], scale=[1, 25, 10, 200], size=(1000, 4))
X_scaled = (X - X.mean(axis=0)) / X.std(axis=0)
```

`X.mean(axis=0)` has shape `(4,)`: one mean per column. It pads to `(1, 4)` and stretches down all 1,000 rows, which is exactly right. Every column ends up with mean 0 and standard deviation 1.

> TIP
> `axis=0` is what makes this work: it collapses the **rows**, leaving one statistic per column. Use the wrong axis and you'd normalise each *sample* against itself, which on a square matrix broadcasts without complaint and means nothing. Chapter 5 is all about `axis=`.

Pairwise distances without a loop

A staple of clustering, nearest neighbours and recommender systems:

```
points = rng.random((5, 2)) * 10
diffs = points[:, None, :] - points[None, :, :]    # (5, 1, 2) - (1, 5, 2) → (5, 5, 2)
distances = np.sqrt((diffs**2).sum(axis=-1))        # (5, 5)
```

Two sanity checks should always hold for a distance matrix, and are worth asserting: the diagonal is zero, and the matrix is symmetric. The notebook also confirms the result matches `scipy.spatial.distance.cdist`.

> FROM THE HISTORY OF DATA SCIENCE
>
> #### The nearest neighbour, 1951
>
> In 1951 two statisticians at the University of California, Berkeley, Evelyn Fix and Joseph Hodges, wrote a technical report for the US Air Force School of Aviation Medicine on a strikingly simple idea: to classify a new observation, look at the training points closest to it and take a vote. It needed no assumptions about the shape of the data, only a way to measure distance. The **k-nearest-neighbours** rule became one of the most widely used methods in machine learning, and its heart is the pairwise distance matrix you just computed in two lines.

One-hot encoding in one line

```
labels = np.array([0, 2, 1, 2, 0, 1])
one_hot = (labels[:, None] == np.arange(3)).astype(np.int8)    # (6, 3)
```

`labels[:, None]` is `(6, 1)`; `np.arange(3)` is `(3,)`. Broadcasting compares every label against every class index, giving exactly one 1 per row.

4.6 The memory trap

Broadcasting doesn't materialise the *inputs*; that's the whole point. But the **result** is fully materialised, and it can be enormous:

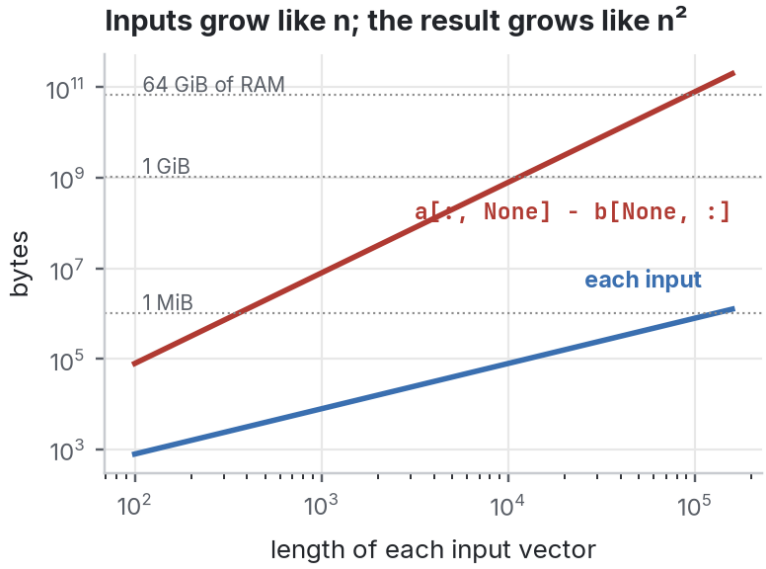


Figure 4.4 Inputs grow linearly; the result of combining every pair grows with the square. Two 10,000-element vectors make an 800 MB matrix; at 100,000 elements, 80 GB.

When you only need a reduction, don't build the intermediate

Algebra first. The sum of all pairwise products, `(a[:, None] * b[None, :]).sum()`, builds an `n × n` matrix only to collapse it. But $\Sigma_i \Sigma_j a_i b_j = (\Sigma_i a_i)(\Sigma_j b_j)$, so `a.sum() * b.sum()` gives the same number with no intermediate at all.

`np.einsum` (Einstein summation) expresses "multiply these axes and sum over that one" in a single pass:

```
np.einsum("ij,jk->ik", A, B)      # matrix product, same as A @ B
np.einsum("ij,ij->i", P, Q)       # row-wise dot products, no (n, m) temporary
```

Chunking, when the big result is genuinely what you need: compute pairwise distances a block of rows at a time, so peak memory is `chunk_size × n` instead of `n²`:

```
def pairwise_distances_chunked(X, chunk_size=500):
    n = X.shape[0]
    out = np.empty((n, n), dtype=X.dtype)
    for start in range(0, n, chunk_size):
        stop = min(start + chunk_size, n)
        block = X[start:stop, None, :] - X[None, :, :]
        out[start:stop] = np.sqrt((block**2).sum(axis=-1))
    return out
```

4.7 Try it yourself

Exercises 4.1 to 4.5 in notebook 04; solutions in notebook 10.

1. **Compatible or not?** Give the result shape or "incompatible" for `(5,4)&(4,)`, `(5,4)&(5,)`, `(5,4)&(5,1)`, `(3,1,4)&(2,4)`, `(2,3,4)&(3,1)`, `(8,1,6)&(7,1)`. Check with `np.broadcast_shapes`.
2. **Fix the silent bug.** `y_true` has shape `(3, 1)` and `y_pred` shape `(3,)`. What shape is `y_true - y_pred`, and what should it be? Fix it and add an assertion.
3. **Normalise rows.** Scale each row of a `(100, 5)` matrix to sum to 1. Explain why `X / X.sum(axis=1)` fails and `X / X.sum(axis=1, keepdims=True)` works.
4. **Distance to centroids.** With `X` of shape `(1000, 3)` and `centroids` of shape `(8, 3)`, compute a `(1000, 8)` distance matrix by broadcasting, then each point's nearest centroid. That's one full iteration of k-means, which you'll reuse in the project.
5. **Avoid the intermediate.** `((a[:, None] - b[None, :])**2).sum()` needs 20 GB for 50,000-element vectors. Expand the algebra and compute the same number using only `sum`, `dot` and scalars.

KEY TAKEAWAYS

- **Align right, pad left with 1s; dimensions must match or be 1.** That's all of broadcasting.
- `np.broadcast_shapes` checks compatibility without allocating.
- `arr[:, None]` **makes a column;** `arr[None, :]` **makes a row.** Say which you mean.
- The dangerous case **succeeds with the wrong shape:** `(n,)` meeting `(n, 1)` gives `(n, n)` and no error.
- **Assert your shapes** in any function that computes a metric or a loss.
- Broadcasting doesn't copy the inputs, but **it does materialise the result.**
- When you only need a reduction, use **algebra**, `einsum` or **chunking.**

CHAPTER 05

05

Summarise, Select, Decide

axis= demystified, boolean masks, and loops you no longer need

Notebook 05_aggregations_and_masking.ipynb · About 90 minutes

IN THIS CHAPTER

- `axis=` demystified, with the one mental model that makes it obvious.
- `keepdims`, and why it exists.
- Boolean masks: the vectorised replacement for an `if` inside a loop.
- `np.where`, `np.select`, `np.clip`: conditional logic without branching.
- Fancy indexing, `argsort` and `argpartition`.
- Counting and grouping: `unique`, `bincount`, `histogram`.

Most of what an analyst does with numbers comes down to three verbs: *summarise* them (a total, an average, a spread), *select* some of them (only the paid customers, only the outliers), and *decide* something per value (clip it, label it, replace it). This chapter does all three without a single Python loop.

5.1 `axis=`: the only rule you need

Almost everyone gets this backwards at first, because “axis 0” sounds like “do it to the rows”. The correct reading is the opposite:

`axis` names the dimension that disappears.

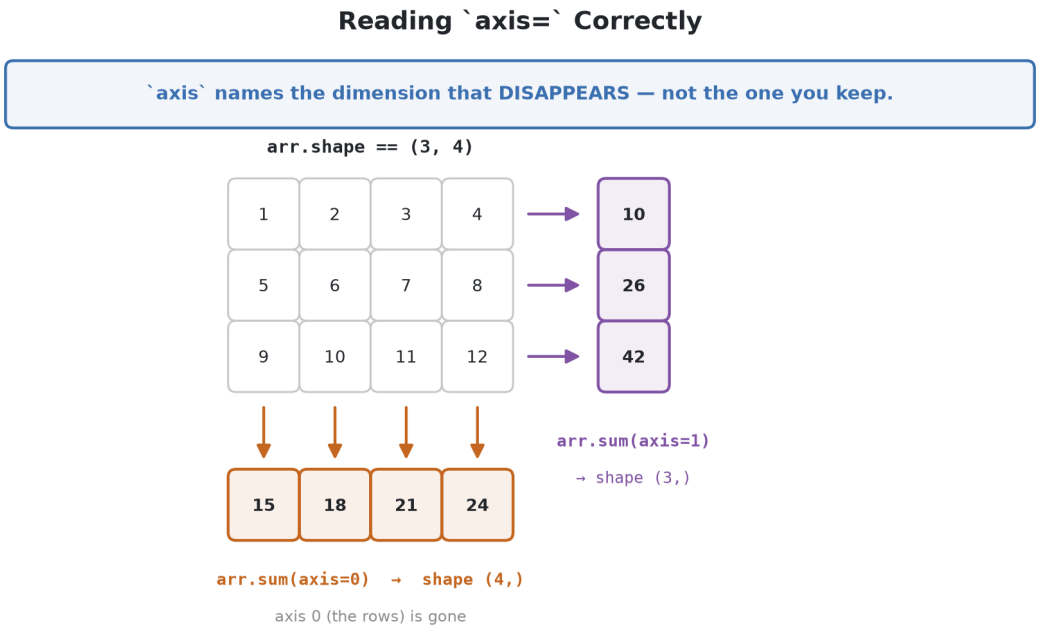


Figure 5.1 arr.sum(axis=0) collapses axis 0, the rows, leaving one value per column. axis=1 collapses the columns, leaving one value per row.

```
arr = np.arange(1, 13).reshape(3, 4)
arr.sum()           # 78: everything collapses to a scalar
arr.sum(axis=0)     # [15 18 21 24]  shape (4,): the rows vanished
arr.sum(axis=1)     # [10 26 42]     shape (3,): the columns vanished
```

The shape arithmetic makes it mechanical: start from the shape tuple, **delete the entry at position** `axis`, and that's your result shape. On a `(2, 3, 4)` cube, `axis=0` gives `(3, 4)`, `axis=1` gives `(2, 4)`, `axis=2` gives `(2, 3)`, and `axis=(0, 1)` gives `(4,)`.

The reductions worth knowing

`sum`, `mean`, `std`, `var`, `min`, `max`, `median`, `percentile`, and the less famous `ptp` ("peak to peak", `max - min`) all take `axis=`.

> **TIP**
> A `ptp` of 0 tells you instantly that a feature is constant, and therefore useless to a model.

`argmin` / `argmax`: where, not what

```
scores = np.array([0.82, 0.91, 0.78, 0.95, 0.88])
models = np.array(["ridge", "lasso", "tree", "forest", "boosting"])
models[scores.argmax()]          # 'forest': argmax returns an INDEX
```

In 2-D, `argmax` flattens the array by default, a classic surprise. `np.unravel_index(m.argmax(), m.shape)` converts the flat index back to `(row, col)`, and `m.argmax(axis=1)` gives the best column per row.

`keepdims`: the argument that saves your broadcasts

A reduction removes a dimension, which then breaks the broadcast you wanted to do next. `keepdims=True` leaves a length-1 dimension in its place:

```
X = rng.random((4, 3))
X.sum(axis=1).shape                    # (4,)
X.sum(axis=1, keepdims=True).shape     # (4, 1)

X / X.sum(axis=1)                      # ValueError: (4,3) vs (4,)
X / X.sum(axis=1, keepdims=True)       # ✔ every row sums to 1
```

WATCH OUT

Without `keepdims`, `(4, 3) / (4,)` raises here, which is lucky. On a **square** matrix the same mistake broadcasts silently and divides by the wrong axis. Use `keepdims=True` whenever a reduction feeds a broadcast.

5.2 Boolean masks

A comparison on an array produces an array of booleans. That **mask** can then select, count or replace, with no loop and no `if`:

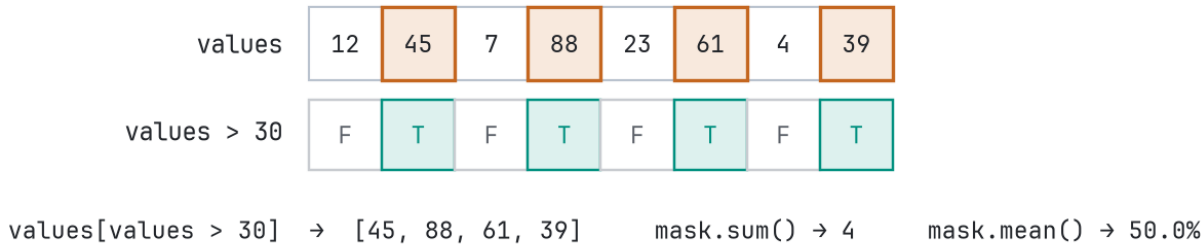


Figure 5.2 A comparison produces a mask; the mask selects. Because True counts as 1, the mask's sum is a count and its mean is a proportion.

```
values = np.array([12, 45, 7, 88, 23, 61, 4, 39])
mask = values > 30
values[mask]       # [45 88 61 39]   a COPY, not a view
mask.sum()         # 4               True is 1, so sum counts
mask.mean()        # 0.5             and mean gives the share
```

Combining conditions: mind the parentheses

NumPy uses `&` (and), `|` (or) and `~` (not) for element-wise logic. Python's `and`, `or` and `not` **don't work:** they try to treat the whole array as one truth value, raising the same ambiguity error you met in Module 01.

```
values[(values > 20) & (values < 60)]     # [45 23 39]
values[(values < 10) | (values > 80)]     # [ 7 88  4]
values[~mask]                             # the complement
```

> **WATCH OUT**
> **`&` binds more tightly than `>` in Python.** Without parentheses, `values > 20 & values < 60` is parsed as `values > (20 & values) < 60`, which raises or, worse, computes something else entirely. Always wrap each comparison in parentheses.

`any` and `all`

```
readings = np.array([[1.0, 2.0, np.nan], [4.0, 5.0, 6.0]])
np.isnan(readings).any()            # is there any NaN at all?
bad_rows = np.isnan(readings).any(axis=1)
readings[~bad_rows]                 # keep only complete rows
```

Assigning through a mask

```
temperatures = np.array([18.0, -999.0, 22.5, -999.0, 25.1])
cleaned = temperatures.copy()
cleaned[cleaned == -999.0] = np.nan     # -999 = "sensor failed" sentinel
np.nanmean(cleaned)                     # 21.87, not the naive -386.48
```

Sentinel values like −999 are a real and common convention. Averaging them as if they were temperatures is catastrophically wrong, and the notebook shows by how much.

5.3 Decisions without branching

`np.where`: a vectorised if/else

```
values = np.array([-5, 3, -2, 8, 0, -1])
np.where(values < 0, 0, values)                    # ReLU in one line
np.where(values > 0, "positive", "non-positive")   # all three arguments broadcast
np.where(values < 0)[0]                            # one argument: the INDICES
```

`np.select`: more than two branches

```
scores = np.array([95, 72, 58, 88, 41, 67])
np.select([scores >= 90, scores >= 70, scores >= 60], ["A", "B", "C"], default="F")
# ['A' 'B' 'F' 'B' 'F' 'C']
```

Conditions are checked **in order**; the first match wins.

`np.clip`: keep values inside a range

```
np.clip(raw, 0, 100)       # bound both ends
np.clip(raw, 0, None)      # lower bound only
```

Clipping at *percentiles* rather than fixed values is called **winsorising**, the technique behind the `OutlierClipper` estimator you wrote in Module 01:

```
low, high = np.percentile(data, [1, 99])
winsorised = np.clip(data, low, high)
```

5.4 Fancy indexing

Indexing with an array of **integers** picks arbitrary elements in any order, repeats allowed. Unlike slicing, it always returns a **copy**.

```
arr = np.array([10, 20, 30, 40, 50])
arr[[0, 2, 4]]      # [10 30 50]
arr[[4, 4, 0]]      # [50 50 10]  repeats and any order

matrix = np.arange(1, 13).reshape(3, 4)
matrix[[0, 1, 2], [3, 2, 1]]    # pairs up: (0,3), (1,2), (2,1) → [4 7 10]
```

Selecting whole rows with a permutation is how you shuffle a dataset, and using the *same* permutation for features and labels keeps them aligned:

```
perm = rng.permutation(len(y))
X_shuffled, y_shuffled = X[perm], y[perm]
```

`argsort`: sort one array by another

```
order = revenue.argsort()[::-1]        # indices, largest revenue first
for rank, i in enumerate(order, start=1):
    print(rank, names[i], revenue[i])
```

`argpartition`: the top k without a full sort

When you only need the largest few, sorting everything is wasted work. `argpartition` is **O(n)** where `argsort` is **O(n log n)**. On two million values, picking the top 10 is many times faster:

```
top_k = np.argpartition(large, -10)[-10:]   # the 10 largest, in no particular order
```

5.5 Counting and grouping

```
values, counts = np.unique(channels, return_counts=True)   # NumPy's Counter
np.bincount(channels)              # faster, for small non-negative integers

categories, encoded = np.unique(labels, return_inverse=True)
categories[encoded]                # label encoding that round-trips exactly
```

A group-by from first principles

`bincount` with `weights` sums the weights that fall in each bin, which is a **group-by sum in one call.** Understanding this makes pandas' `groupby` (Module 03) far less mysterious:

```
group_ids = np.array([0, 1, 0, 2, 1, 0, 2])
amounts = np.array([10.0, 20.0, 5.0, 8.0, 12.0, 3.0, 15.0])

group_sums = np.bincount(group_ids, weights=amounts)   # [18. 32. 23.]
group_counts = np.bincount(group_ids)                  # [3 2 2]
group_means = group_sums / group_counts
```

The notebook cross-checks the result against pandas, and it matches.

> FROM THE HISTORY OF DATA SCIENCE
>
> **The nurse who grouped by cause**
>
> In the Crimean War of the 1850s, Florence Nightingale went to the British Army hospital at Scutari and kept meticulous records of how soldiers died. When she grouped the deaths **by cause and by month**, the picture was damning: far more men were dying of preventable diseases (cholera, typhus, dysentery) than of their wounds. She presented the aggregated figures to the government in 1858 with her now-famous "rose" diagrams, and they helped drive sweeping sanitary reforms. She later became the first woman elected a Fellow of the Royal Statistical Society. Her method was, at heart, a `groupby` followed by a well-chosen chart.

Binning with `digitize` and `histogram`

```
bin_edges = np.array([18, 30, 45, 60, 100])
counts, _ = np.histogram(ages, bins=bin_edges)      # how many per age band
np.digitize(ages, bin_edges)                        # which band each age is in
```

5.6 Putting it together: a vectorised quality report

The notebook combines everything above into one function that profiles a numeric feature matrix with no loops at all. For each column it reports the share missing, mean, standard deviation, min, median, max, the share of outliers by the **1.5 × IQR rule**, and whether the column is constant:

```
q1, q3 = np.nanpercentile(X, [25, 75], axis=0)
iqr = q3 - q1
is_outlier = (X < q1 - 1.5 * iqr) | (X > q3 + 1.5 * iqr)
outlier_share = np.nansum(is_outlier, axis=0) / len(X) * 100
```

Run on a deliberately messy matrix (one well-behaved column, one skewed, one constant, one with injected missing values), it flags each problem immediately.

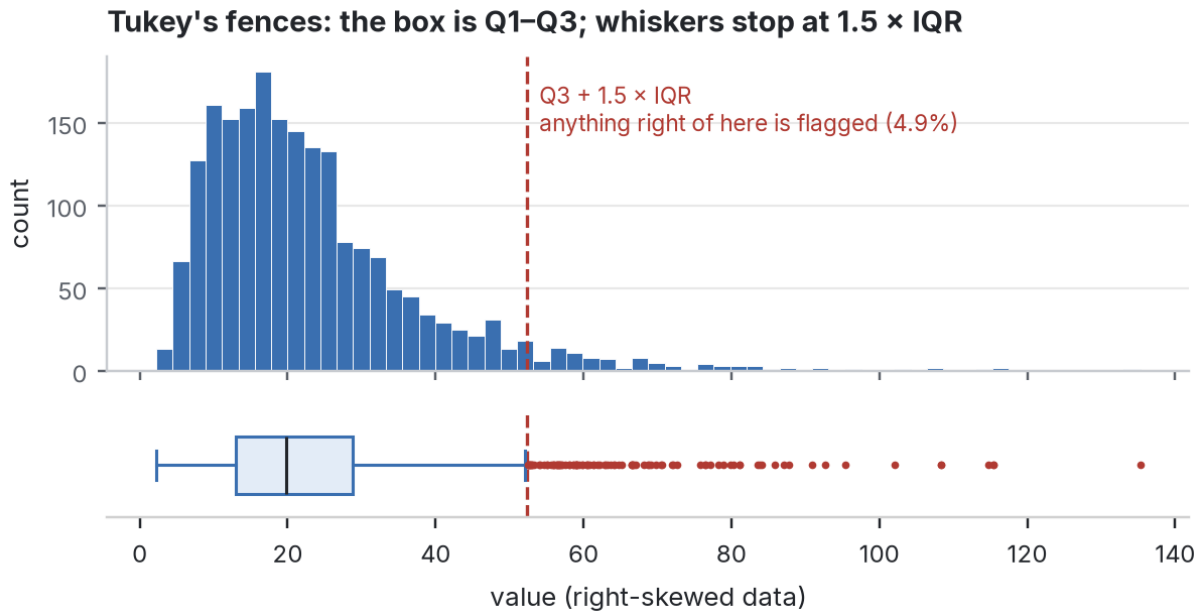


Figure 5.3 The 1.5 × IQR rule on right-skewed data: a box from the first to the third quartile, whiskers out to 1.5 times its width, and everything beyond flagged.

> NOTE
>
> The 1.5 × IQR fences come from the American statistician **John Tukey**, who popularised the box plot in his 1977 book *Exploratory Data Analysis*. Tukey argued that looking at data should come *before* testing hypotheses about it, an idea so influential that "EDA" is now a stage of every data science project. He also coined the word "bit".

Notice what the chart shows, though: on skewed data the rule flags plenty of perfectly genuine values in the long tail. A fence is a prompt to look, not a licence to delete.

5.7 Try it yourself

Exercises 5.1 to 5.5 in notebook 05; solutions in notebook 10.

1. **Axis drill.** For `a = np.arange(24).reshape(2, 3, 4)`, give the shapes of `a.sum(axis=0)`, `a.sum(axis=1)`, `a.sum(axis=2)`, `a.sum(axis=(0, 2))`, `a.mean(axis=1, keepdims=True)` and `a.max(axis=-1)` before running them.
2. **The keepdims trap.** For a `(5, 5)` matrix both `X / X.sum(axis=1)` and `X / X.sum(axis=1, keepdims=True)` run. Show the first is wrong and explain exactly what it computed.
3. **Masking practice.** For `rng.normal(100, 20, 10_000)`: count values beyond two standard deviations, replace them with the mean without a loop, and report the share inside `[80, 120]`.
4. **Group-by from scratch.** `group_stats(keys, values)` returns count, sum, mean and max per key using only NumPy, handling string keys via `np.unique(..., return_inverse=True)`. Check it against pandas.

5 **Top-k per row.** For `scores` of shape `(1000, 50)`, find each user's top 5 items with `argpartition`, returning a `(1000, 5)` array sorted by score. That's the core of a recommender's serving path.

KEY TAKEAWAYS

- `axis` **names the dimension that disappears.** Delete it from the shape tuple to get the result shape.
- `keepdims=True` whenever a reduction feeds a broadcast.
- `argmin`/`argmax` return **indices**, and flatten in 2-D unless you pass an axis.
- Boolean masks replace `if` inside a loop. Use `&`, `|`, `~`, and **parenthesise** every comparison.
- `mask.sum()` counts; `mask.mean()` gives the proportion.
- `np.where` is vectorised if/else; `np.select` handles many branches; `np.clip` bounds a range.
- **Fancy indexing returns a copy**, with repeats and any order allowed.
- `argpartition`, not `argsort`, when you only need the top k.
- `np.bincount(ids, weights=values)` is a group-by sum in one call.

CHAPTER 06

06

Linear Algebra Without Tears

Dot products, solving systems, least squares, eigenvectors and SVD

Notebook 06_linear_algebra.ipynb · About 90 minutes

IN THIS CHAPTER

- Vectors: dot products, norms, and what "similarity" means geometrically.
- Matrix multiplication: `@` versus `*`, and why the difference matters.
- Solving linear systems, and why **you should never invert a matrix**.
- Least squares: deriving linear regression from scratch.
- Eigenvalues and SVD, the machinery behind PCA.
- Condition numbers: recognising a problem that is numerically hopeless.

Linear algebra has a fearsome reputation, mostly earned in lecture halls full of proofs. For a data scientist it's something friendlier: a small set of operations on tables of numbers, each with a clear geometric meaning, all of which NumPy performs for you. Every model in Modules 11 to 19 is linear algebra underneath, so this is the chapter that pays off longest.

6.1 Vectors

The dot product

$a \cdot b = \Sigma\, a_i b_i$. Geometrically it equals $|a|\,|b| \cos\theta$: it measures how much two vectors point the same way.

```
a = np.array([3.0, 4.0]); b = np.array([1.0, 2.0])
a @ b              # 11.0, the preferred syntax
np.dot(a, b)       # the same
(a * b).sum()      # the same, spelled out
```

Norms: measuring length

```
v = np.array([3.0, 4.0])
np.linalg.norm(v)              # 5.0, Euclidean (L2): the 3-4-5 triangle
np.linalg.norm(v, ord=1)       # 7.0, Manhattan (L1)
np.linalg.norm(v, ord=np.inf)  # 4.0, the largest component
```

```
np.linalg.norm(X, axis=1)      # one length per row: the everyday case
```

> **NOTE**
> L1 and L2 are exactly the penalties in Lasso and Ridge regression (Module 11). L1's sharp corners are what push coefficients to exactly zero, which gives you feature selection; L2's smooth ball merely shrinks them.

Cosine similarity

Divide out the lengths and only the *direction* remains. This is how text embeddings, recommender systems and nearest-neighbour search compare items:

```
def cosine_similarity(u: np.ndarray, v: np.ndarray) -> float:
    denominator = np.linalg.norm(u) * np.linalg.norm(v)
    if denominator == 0:
        return 0.0              # the angle is undefined; 0 misleads least
    return float(u @ v / denominator)
```

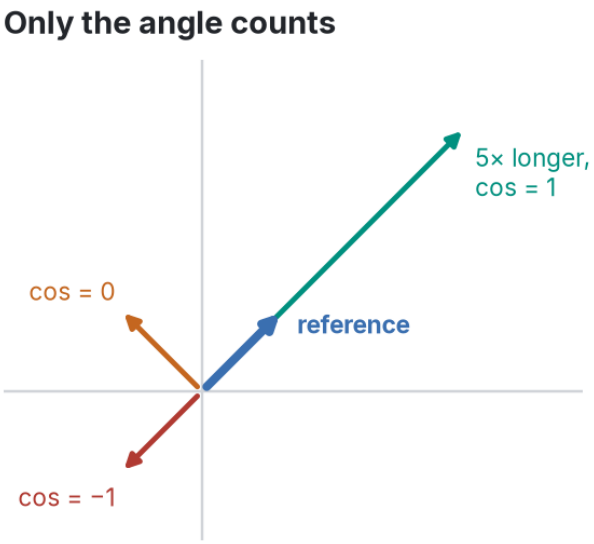


Figure 6.1 Cosine similarity ignores length: a vector five times longer in the same direction scores 1, a perpendicular one 0, an opposite one −1.

6.2 Matrix multiplication

`@` is not `*`

This is the most common source of wrong answers for people arriving from MATLAB or from a maths background:

```
A = np.array([[1, 2], [3, 4]]); B = np.array([[5, 6], [7, 8]])
A * B     # [[ 5 12] [21 32]]   element-wise
A @ B     # [[19 22] [43 50]]   the matrix product
```

The shape rule

`(m, n) @ (n, p) → (m, p)`. The **inner dimensions must match**, and they vanish; the outer ones survive. `(3, 4) @ (4, 2)` gives `(3, 2)`; `(2, 3) @ (2, 3)` is an error.

1-D arrays adapt, which is convenient and risky

A 1-D array is treated as a column when it's on the right and as a row when it's on the left, and `v @ v` gives a plain scalar:

```
M = np.array([[1, 2, 3], [4, 5, 6]])     # (2, 3)
M @ np.array([1, 1, 1])                  # [ 6 15]  shape (2,)
np.array([1, 1]) @ M                     # [5 7 9]  shape (3,)
```

> **TIP**
> Because 1-D arrays bend to whichever orientation makes the product work, a genuine shape mistake can slip through. When shapes matter, be explicit with `reshape(-1, 1)`.

Batched multiplication

`@` works on the **last two axes** and broadcasts everything before them, which is exactly how a neural network processes a batch: a `(32, 4, 3)` batch `@` a `(3, 5)` weight matrix gives `(32, 4, 5)`, thirty-two independent products in one call.

6.3 Solving linear systems

Never invert a matrix

To solve `Ax = b`, the algebra says `x = A⁻¹b`. **Don't compute it that way.** `np.linalg.solve` uses an LU decomposition: faster, and markedly more accurate.

```
A = np.array([[3.0, 1.0], [1.0, 2.0]]); b = np.array([9.0, 8.0])
x = np.linalg.solve(A, b)       # [2. 3.]
np.allclose(A @ x, b)           # True
```

On a nice small system both methods agree. On an awkward one they don't. The notebook builds a 12 × 12 **Hilbert matrix** (entries 1/(i+j+1)), a textbook example of a nearly singular matrix, and solves a system whose true answer is all ones. `solve` gets close; `inv(A) @ b` is visibly worse.

The condition number

`np.linalg.cond(A)` estimates how much a tiny change in the input can be amplified in the output. A rule of thumb worth remembering: **you lose about `log10(cond)` significant digits.**

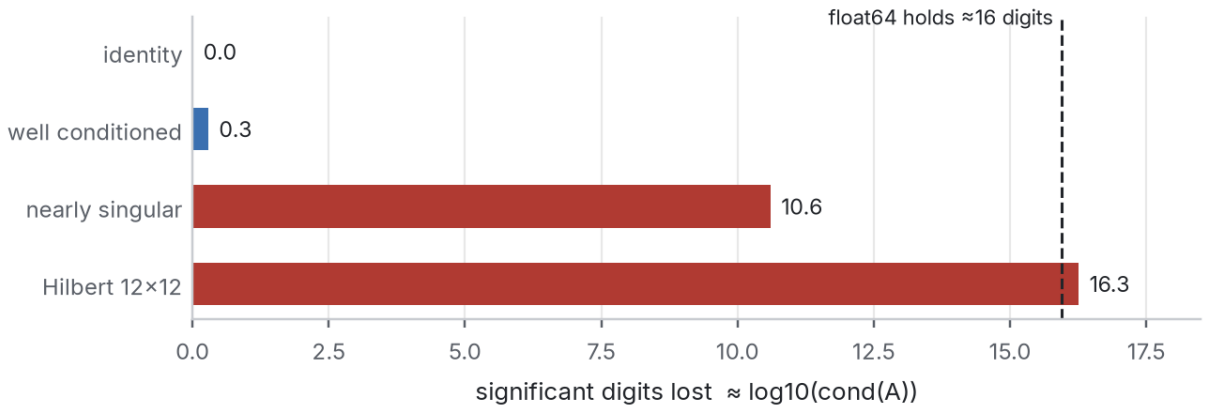


Figure 6.2 Digits lost to the condition number, for four matrices. float64 only has about sixteen to spend; the 12 × 12 Hilbert matrix spends all of them.

> **WATCH OUT**
>
> A high condition number is why you **standardise features** before fitting a linear model, and why perfectly correlated columns break a regression. It isn't a numerical curiosity: it's the reason Ridge regression adds `alpha` to the diagonal (Module 11).

> **NOTE**
>
> The term "condition number" was introduced by **Alan Turing** in a 1948 paper on rounding errors in matrix computations, written while he was thinking about how the first electronic computers would cope with real arithmetic. The codebreaker was also one of the founders of numerical analysis.

6.4 Least squares: linear regression from first principles

With more rows than unknowns there's usually no exact solution. **Least squares** finds the coefficients that minimise the sum of squared residuals, `‖Xβ - y‖²`.

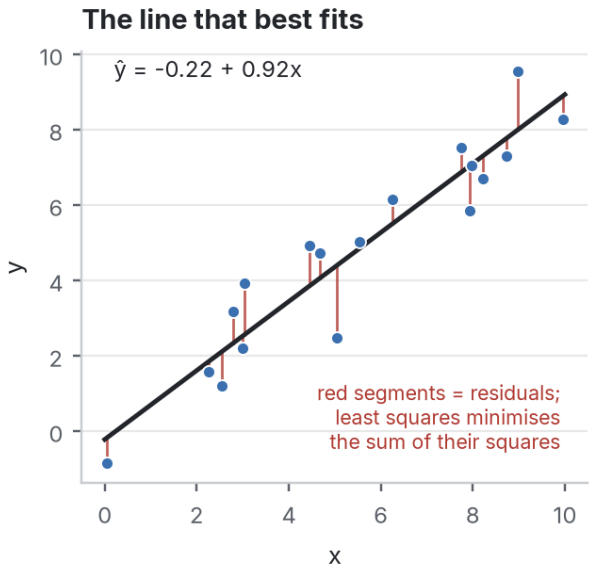


Figure 6.3 Least squares picks the line that makes the red segments, the residuals, as small as possible in total squared length.

The **normal equations** give a closed form, $\hat{\beta} = (X^T X)^{-1} X^T y$. Add a column of ones to `X` and the intercept becomes just another coefficient. The notebook simulates data with known coefficients (intercept 5, slopes 2.5, −1.3 and 0.8) and recovers them three ways:

```
X_design = np.column_stack([np.ones(n_samples), X])

beta_inv = np.linalg.inv(X_design.T @ X_design) @ X_design.T @ y      # literally
beta_solve = np.linalg.solve(X_design.T @ X_design, X_design.T @ y)   # better
beta_lstsq, *_ = np.linalg.lstsq(X_design, y, rcond=None)             # use this
```

All three agree here, because the problem is well conditioned. `lstsq` is still the one to use: it works through the SVD, so it survives **rank deficiency**, two perfectly collinear features, where `solve` raises a `LinAlgError` (“Singular matrix”) and `lstsq` returns the minimum-norm solution, splitting the shared effect between the twins. The notebook confirms our coefficients match scikit-learn’s `LinearRegression` to many decimal places, because it’s the same computation.

FROM THE HISTORY OF DATA SCIENCE

Finding a lost planet with least squares

On New Year’s Day 1801 the astronomer Giuseppe Piazzi spotted a faint new object, later named Ceres. He tracked it for about six weeks before it vanished into the glare of the Sun, and astronomers had too few observations to work out where it would reappear. A 24-year-old Carl Friedrich Gauss took the scattered measurements, fitted an orbit with a method of his own, and predicted where to look. On the last night of 1801, Ceres was found almost exactly where he said. Gauss’s method was **least squares.** Adrien-Marie Legendre published it first, in 1805; Gauss said he’d been using it since 1795, and the two argued about priority for years. Both would recognise `np.linalg.lstsq` at a glance.

6.5 Eigenvalues and eigenvectors

An eigenvector of `A` is a direction that `A` only *stretches*, never rotates: `Av = λv`. The eigenvalue `λ` is the stretch factor.

```
A = np.array([[4.0, 1.0], [2.0, 3.0]])
eigenvalues, eigenvectors = np.linalg.eig(A)     # vectors are the COLUMNS
lam, v = eigenvalues[0], eigenvectors[:, 0]
np.allclose(A @ v, lam * v)                       # True: the definition
```

Why it matters: PCA

Principal Component Analysis is the eigendecomposition of the covariance matrix. The eigenvectors are the principal directions; the eigenvalues are the variance along each one.

```
centred = points - points.mean(axis=0)
cov = (centred.T @ centred) / (len(centred) - 1)
eigenvalues, eigenvectors = np.linalg.eigh(cov)    # eigh: for symmetric matrices
order = eigenvalues.argsort()[::-1]                # eigh returns ascending order
projected = centred @ eigenvectors[:, order]
```

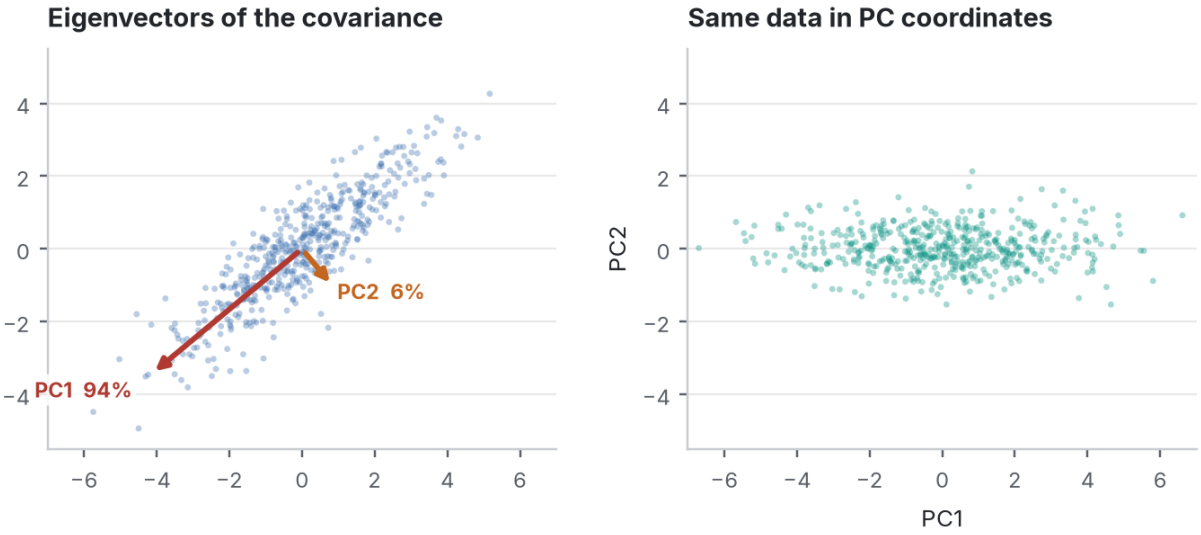


Figure 6.4 Left: correlated data with its two principal axes, scaled by their spread. Right: the same points expressed in principal-component coordinates, where they are uncorrelated.

In the rotated coordinates the covariance matrix is diagonal: the components are uncorrelated, which is the whole point of PCA. The notebook checks the explained-variance ratios against scikit-learn's `PCA`, and they match.

> **NOTE**
> PCA is older than computers. **Karl Pearson** described it in 1901 as finding "lines and planes of closest fit" to a cloud of points, and **Harold Hotelling** developed and named it in 1933. They computed by hand what `eigh` does in microseconds.

6.6 Singular Value Decomposition

The SVD factors **any** matrix (square or not, invertible or not) as `A = U Σ Vᵀ`. It underpins PCA, recommender systems, image compression and `lstsq` itself.

```
U, s, Vt = np.linalg.svd(A, full_matrices=False)
np.allclose(A, U @ np.diag(s) @ Vt)     # True: an exact reconstruction
```

Low-rank approximation

Keeping only the `k` largest singular values gives the **best possible** rank-`k` approximation of the matrix. That's compression, and it's also denoising. The notebook builds a 60 × 40 matrix that is secretly rank 3, plus a little noise:

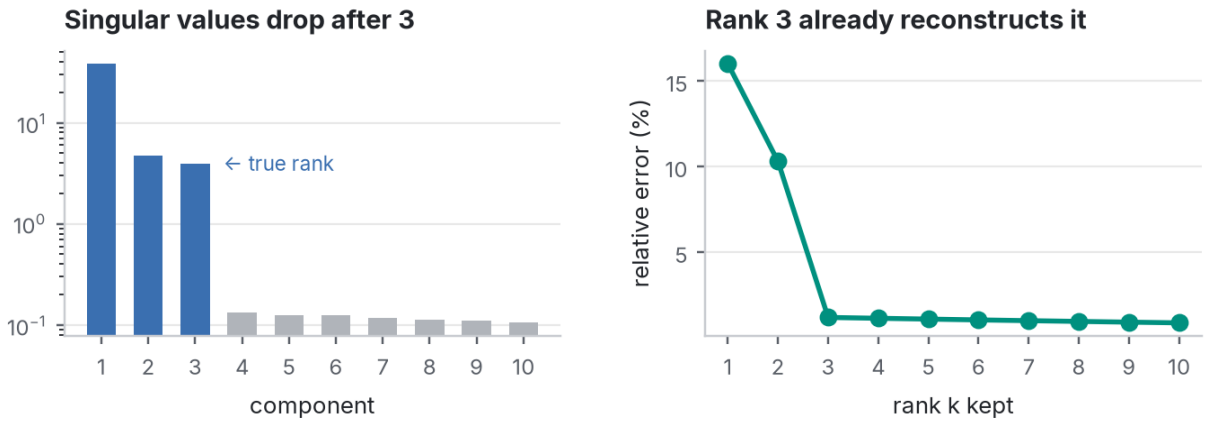


Figure 6.5 The singular values drop sharply after the third, the true rank, and keeping just three components already reconstructs the matrix almost perfectly.

Storing the rank-3 approximation takes `3 × (60 + 40 + 1)` numbers, about an eighth of the original 2,400.

FROM THE HISTORY OF DATA SCIENCE

A million dollars for a better recommendation

In October 2006 Netflix offered **$1,000,000** to anyone who could improve its film-recommendation accuracy by 10%. Thousands of teams competed. One of the most influential early ideas came from a programmer writing as "Simon Funk", who posted on his blog how to factorise the gigantic, mostly empty users × films ratings matrix into a handful of hidden "taste" factors, an SVD-style low-rank approximation like the one above. The prize was finally won in September 2009 by a merged team called *BellKor's Pragmatic Chaos*, whose winning blend still leaned heavily on matrix factorisation.

6.7 Quick reference

Call	Purpose
`a @ b`	dot product / matrix product
`np.linalg.norm(x, axis=1)`	row-wise vector lengths
`np.linalg.solve(A, b)`	solve `Ax = b`, never via `inv()`
`np.linalg.lstsq(A, b)`	least squares; survives rank deficiency
`np.linalg.inv(A)`	inverse: almost always the wrong tool
`np.linalg.det(A)`, `matrix_rank(A)`	determinant, numerical rank
`np.linalg.cond(A)`	condition number: check it before trusting a fit
`np.linalg.eig(A)` / `eigh(A)`	eigendecomposition; `eigh` for symmetric matrices
`np.linalg.svd(A)`	singular value decomposition
`np.trace(A)`, `A.T`	sum of the diagonal; transpose (a view)

6.8 Try it yourself

Exercises 6.1 to 6.5 in notebook 06; solutions in notebook 10.

1. **Shapes.** Result shape or "error": `(3,4)@(4,5)`, `(3,4)@(3,4)`, `(4,)@(4,)`, `(3,4)@(4,)`, `(10,3,4)@(4,2)`, `(3,)@(3,4)`.
2. **Cosine similarity matrix.** For `E` of shape `(1000, 128)`, compute the full `(1000, 1000)` cosine-similarity matrix without a loop. Hint: normalise the rows, then one product does the rest.
3. **Implement least squares.** `fit_linear(X, y, fit_intercept=True)` via `lstsq`, validated against scikit-learn. Then add `alpha` for Ridge, $\beta = (X^\top X + \alpha I)^{-1} X^\top y$, and match `sklearn.linear_model.Ridge`.
4. **Condition number and collinearity.** Build two columns with correlation 0.9999, report $\text{cond}(X^\top X)$, compare normal-equation and `lstsq` coefficients, then standardise and report the condition number again.
5. **Image compression with SVD.** Reconstruct a 200 × 200 structured "image" at ranks 5, 20 and 50 and plot all four versions with the storage each needs.

KEY TAKEAWAYS

- `@` **is matrix multiplication**; `*` is element-wise.
- `(m, n) @ (n, p) → (m, p)`: inner dimensions match and disappear.
- **Never use** `inv` **to solve a system.** `solve` is faster and more accurate; `lstsq` also survives rank deficiency.
- Least squares *is* linear regression, and `lstsq` is what scikit-learn uses.
- `cond(A)` tells you how many digits a problem will cost: that's why you standardise, and why Ridge exists.
- **PCA is the eigendecomposition of the covariance matrix.** Use `eigh` for symmetric matrices.
- **SVD works on any matrix** and gives the best low-rank approximation: compression, denoising, recommenders.

CHAPTER 07

07

Dice You Can Replay

Reproducible randomness, Monte Carlo, and the bootstrap

Notebook 07_random_and_simulation.ipynb · About 75 minutes

IN THIS CHAPTER

- The modern `Generator` API, and why the legacy `np.random.seed` is a liability.
- Reproducibility that survives parallelism, via `SeedSequence` and `spawn`.
- The distributions you'll actually draw from, and when each one applies.
- Sampling: with and without replacement, weighted, permutations.
- **Monte Carlo:** answering questions that have no formula.
- **The bootstrap:** confidence intervals without distributional assumptions.

Computers are deterministic machines, so the "random" numbers they produce are really an extremely long, extremely well-shuffled sequence that starts from a seed. That's not a weakness; it's a superpower. The same seed replays the same dice, which means a simulation can be checked, debugged and reproduced by anyone. This chapter is the bridge into the statistics half of the course: everything in Modules 07 to 09 either uses these tools or is checked against them.

7.1 The `Generator` API

NumPy has two random systems. Use the newer one.

```
np.random.seed(42); np.random.rand(3)       # ✘ legacy: one hidden global state
rng = np.random.default_rng(seed=42)        # ✔ modern: an object you own
rng.random(3)
```

Why the global state is a problem

`np.random.seed` sets state that *every* library in the process shares. If scikit-learn, a plotting call or a colleague's helper draws a number between your seed and your draw, your "reproducible" result silently changes:

```
np.random.seed(42); first = np.random.rand(3)
np.random.seed(42); third_party_function(); second = np.random.rand(3)
```

```
np.array_equal(first, second)     # False: reproducibility broken
```

A `Generator` keeps its state privately, inside the object, so nothing else can disturb it.

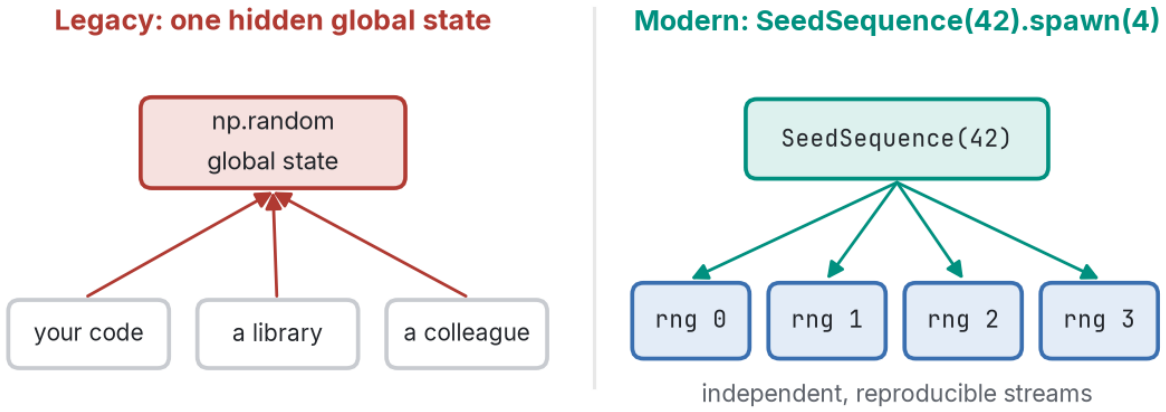


Figure 7.1 Left: every caller draws from one shared global state. Right: a parent SeedSequence spawns independent, reproducible streams, one per worker.

TIP

Create one `Generator` at the top of your script or notebook and pass it down. Functions that need randomness should take `rng` as a parameter rather than reaching for a global, the same "no hidden state" principle as Module 01's pure functions:

```
def simulate_returns(rng: np.random.Generator, n_days: int = 252):
    return rng.normal(loc=0.0005, scale=0.01, size=n_days)
```

Independent streams for parallel work

Giving each worker process `seed + i` is a genuine mistake: nearby seeds can produce correlated streams. `SeedSequence` spawning gives provably independent children, and re-spawning from the same parent reproduces them exactly:

```
children = np.random.SeedSequence(42).spawn(4)
workers = [np.random.default_rng(child) for child in children]
```

FROM THE HISTORY OF DATA SCIENCE

A best-seller made of random digits

Before computers could generate random numbers cheaply, scientists bought them by the book. In 1955 the RAND Corporation published *A Million Random Digits with 100,000 Normal Deviates*, produced by an electronic "roulette wheel" and then carefully tested and corrected for bias. It was used for decades to design experiments and run simulations by hand. Its pages are about as unpredictable as reading gets, and they inspired a long-running internet joke of tongue-in-cheek reader reviews. Today one line, `rng = np.random.default_rng(42)`, gives you more digits than RAND could print, and the same ones every time.

7.2 The distributions you'll use

Each distribution models a different kind of process. Choosing one is a **modelling decision**, not a technical one.

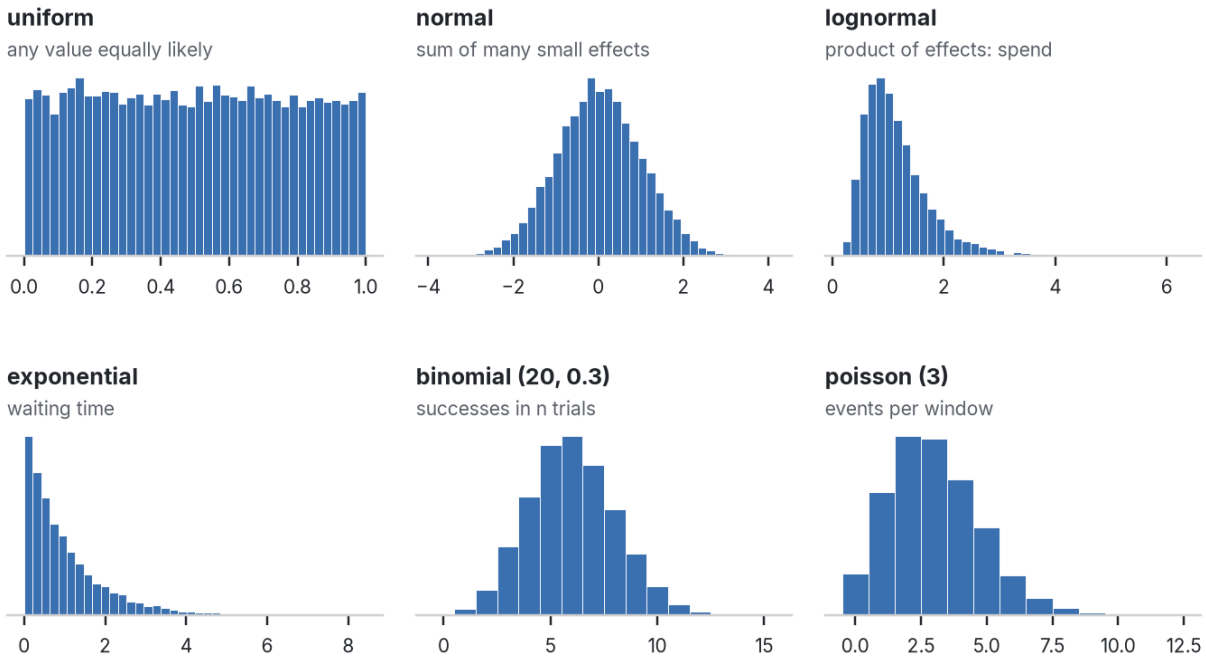


Figure 7.2 Six distributions you'll draw from constantly, each with the kind of process it describes.

Distribution	Models	You'll meet it in
Uniform	any value in a range, equally likely	random initialisation, shuffling
Normal	sums of many small effects	measurement error, residuals
Lognormal	quantities that are *products* of effects	income, spend, session length
Exponential	waiting time between events	churn, time to failure
Binomial	successes in n independent trials	conversions out of visitors
Poisson	count of events in a fixed window	arrivals per hour, defects per batch

> NOTE
> Customer spend is **lognormal**, not normal, which is why the mean sat well above the median in Module 01's very first analysis. Recognising the generating process tells you which summary statistic is honest.

7.3 Sampling

```
population = np.array(["A", "B", "C", "D", "E"])
rng.choice(population, 8)                          # with replacement
rng.choice(population, 4, replace=False)           # without
rng.choice(population, 8, p=[0.6, 0.2, 0.1, 0.05, 0.05])   # weighted

rng.permutation(arr)    # returns a shuffled COPY
rng.shuffle(arr)        # shuffles IN PLACE and returns None
```

A train/test split is just a permutation cut in two, which is what scikit-learn does for you:

```
indices = rng.permutation(n_samples)
split = int(0.8 * n_samples)
train_idx, test_idx = indices[:split], indices[split:]
```

7.4 Monte Carlo

When a quantity is hard to derive, **simulate it:** draw many random outcomes and average. The estimate converges at a rate of `1/√n`, which is slow, but it works for problems where no formula exists at all.

The classic warm-up estimates π. Scatter points uniformly over a unit square; the fraction landing inside the quarter circle approaches π/4:

```
def estimate_pi(rng: np.random.Generator, n_points: int) -> float:
    points = rng.random((n_points, 2))
    inside = (points**2).sum(axis=1) <= 1.0
    return 4.0 * inside.mean()
```

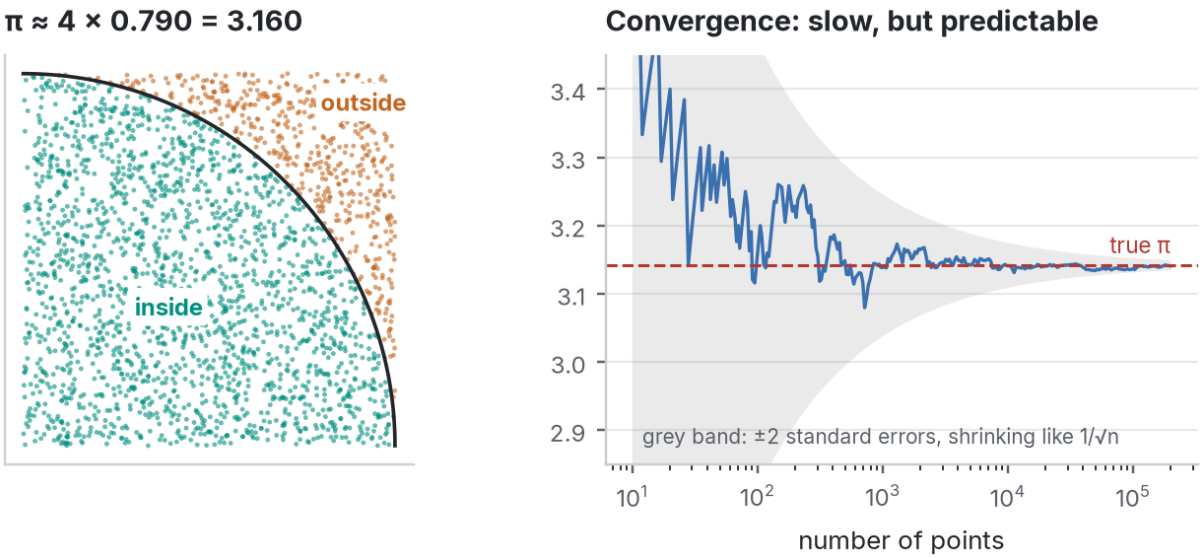


Figure 7.3 Left: what the simulation does. Right: the running estimate wobbling towards π, inside a band of ±2 standard errors that narrows like 1/√n.

That `1/√n` is the catch: **a hundred times more points buys only ten times more accuracy.**

FROM THE HISTORY OF DATA SCIENCE

A card game, a hospital bed, and the hydrogen bomb

In 1946 the mathematician Stanislaw Ulam was recovering from an illness and passing the time with solitaire. He wondered what the chance was of a game coming out, tried to calculate it, gave up, and realised he could simply *play many games and count*. He took the idea to John von Neumann, and the two applied it to neutron diffusion problems at Los Alamos that had no tractable formula, running the calculations on ENIAC, one of the first electronic computers. Their colleague Nicholas Metropolis suggested the code name **Monte Carlo**, after the casino where Ulam's uncle liked to gamble. Your `estimate_pi` is a direct descendant.

NOTE

Estimating π by chance is far older still. In 1777 the French naturalist Georges-Louis Leclerc, Comte de Buffon, analysed the probability that a needle dropped on a floor of parallel boards crosses a line, a probability that involves π. "Buffon's needle" is often called the first Monte Carlo experiment.

Checking simulation against a formula

"A checkout page converts at 12%. If 40 people visit, what's the probability that at least 8 convert?" This one does have a closed form, which makes it a good test: half a million simulated visits agree with the exact binomial answer to about three decimal places, just as `1/√n` predicts.

```
conversions = rng.binomial(40, 0.12, 500_000)
simulated = (conversions >= 8).mean()
exact = 1 - stats.binom.cdf(7, 40, 0.12)
```

7.5 The bootstrap

The bootstrap estimates the uncertainty of *any* statistic by **resampling the data you already have, with replacement.** It needs **no assumption** about the underlying distribution, which is why it works for medians, ratios and correlations where classical formulas are awkward or don't exist.

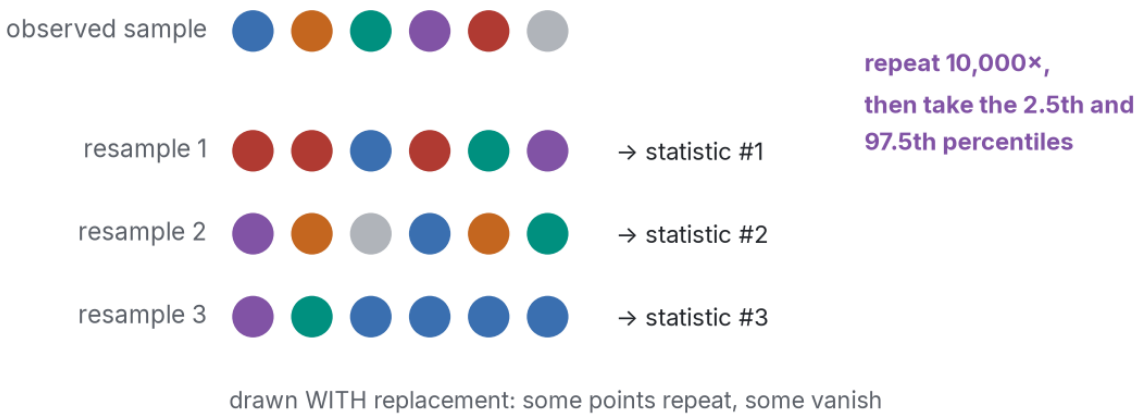


Figure 7.4 One observed sample, resampled with replacement: some points appear twice, some vanish. Compute the statistic on each resample, thousands of times, and read off the percentiles.

The notebook's `bootstrap_ci` draws all the resamples in one vectorised call instead of a loop:

```
def bootstrap_ci(data, statistic=np.mean, *, n_resamples=10_000,
                 confidence=0.95, rng=None):
    rng = rng or np.random.default_rng()
    n = len(data)
    resample_idx = rng.integers(0, n, size=(n_resamples, n))   # (B, n) at once
    estimates = statistic(data[resample_idx], axis=1)
    alpha = 1 - confidence
    low, high = np.percentile(estimates, [100 * alpha / 2, 100 * (1 - alpha / 2)])
    return float(statistic(data)), float(low), float(high)
```

On 300 right-skewed spend values, it gives a 95% interval for the **mean** and, just as easily, for the **median**, which has no simple closed-form standard error. The bootstrap doesn't care. The notebook confirms the interval matches SciPy's own `scipy.stats.bootstrap` to within 1%.

> NOTE
>
> Plot the bootstrap distribution of the *mean* and it looks close to normal, even though the data itself is strongly skewed. That's the **Central Limit Theorem** appearing on its own; Module 07 develops it properly.

> FROM THE HISTORY OF DATA SCIENCE
>
> ### Pulling yourself up by your bootstraps
>
> In 1979 the Stanford statistician Bradley Efron published "Bootstrap methods: another look at the jackknife". The name comes from the old expression about pulling yourself up by your own bootstraps, an impossible feat, because the method seems to get something from nothing: uncertainty estimates from a single sample, without new data and without a formula. What made it practical was cheap computing. A calculation that would have taken a 1950s statistician weeks became a few seconds. Efron received the International Prize in Statistics in 2018, largely for this idea.

7.6 Try it yourself

Exercises 7.1 to 7.5 in notebook 07; solutions in notebook 10.

1. **Break and fix reproducibility.** Show that an unrelated function called between `np.random.seed` and a draw changes the result; rewrite with an injected `Generator` and show it can't happen.
2. **Choose the distribution.** Name and justify one for: support tickets per hour; time between two tickets; whether each of 500 emails is opened; yearly revenue per customer; the error of a well-calibrated sensor.
3. **The birthday problem.** Simulate the chance that at least two people in a group of `k` share a birthday, for `k` from 2 to 60. Find the smallest `k` above 50% and compare with the exact formula.
4. **Bootstrap a difference.** For two groups of spend, bootstrap a 95% interval for the **difference in medians**. Does it contain zero, and what does that let you claim, and not claim?
5. **Sample size by simulation.** A page converts at 10%. Find the sample size per arm needed to detect a lift to 11% with 80% power at α = 0.05, by simulating 2,000 A/B tests per candidate size. That's the core of Module 09.

KEY TAKEAWAYS

- Use `np.random.default_rng(seed)`. The legacy global state makes reproducibility fragile.
- **Inject the generator** into functions rather than reaching for a global.
- For parallel work use `SeedSequence.spawn`, never `seed + i`.
- Match the **distribution to the process**: counts → Poisson, waiting times → exponential, multiplicative quantities → lognormal.
- **Monte Carlo** answers questions with no formula; its error shrinks like **1/√n**.
- **The bootstrap** gives an interval for any statistic, with no distributional assumption: resample with replacement and take percentiles.
- Vectorise the resampling: `rng.integers(0, n, size=(B, n))`.

CHAPTER 08

08

Under the Hood

Strides, memory order, temporaries, and knowing when to change tools

Notebook 08_performance_and_memory.ipynb · About 75 minutes

IN THIS CHAPTER

- **Strides:** how one flat buffer pretends to be an N-dimensional array.
- C order versus Fortran order, and why walking the "wrong" way is slower.
- In-place operations, and how to stop allocating temporaries.
- Measuring memory honestly, hidden intermediates included.
- `np.memmap` for arrays bigger than your RAM.
- Knowing when NumPy has run out of road, and what comes next.

So far we've treated an array as a grid of numbers. This chapter lifts the lid and shows what's really there: one long, flat strip of bytes, plus a tiny recipe for reading it as a grid. Once you can see the strip, a dozen performance puzzles solve themselves.

8.1 Strides: the trick behind everything

An ndarray is a **flat buffer** plus a rule for interpreting it. That rule is the `strides` tuple: how many **bytes** to step to move one position along each axis.

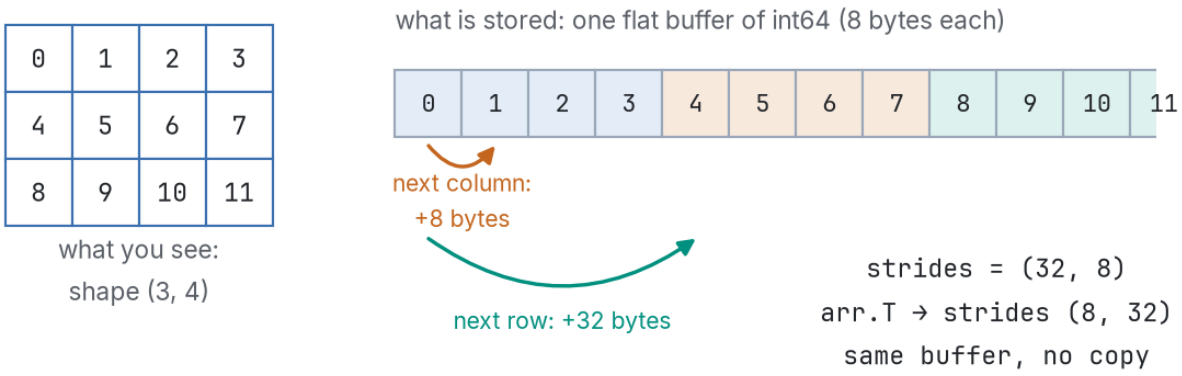


Figure 8.1 What you see is a 3 × 4 grid; what's stored is twelve int64 values in a row. Moving one column steps 8 bytes; moving one row steps 32.

```
arr = np.arange(12, dtype=np.int64).reshape(3, 4)
arr.strides      # (32, 8): next row = skip 32 bytes (4 elements), next column = 8
arr.T.strides    # (8, 32): swapped. Same buffer, no data moved
arr[::2].strides # (64, 8): skipping rows just doubles the row stride
```

This is why reshaping and transposing are **free**: they only rewrite the strides, never the data. It's also why so many operations return views (Chapter 2): a view is just a new set of strides over the same strip of memory.

C order versus Fortran order

There are two natural ways to lay a grid out in a strip. **C order** (row-major, NumPy's default) stores row 0, then row 1, and so on. **Fortran order** (column-major) stores column 0, then column 1.

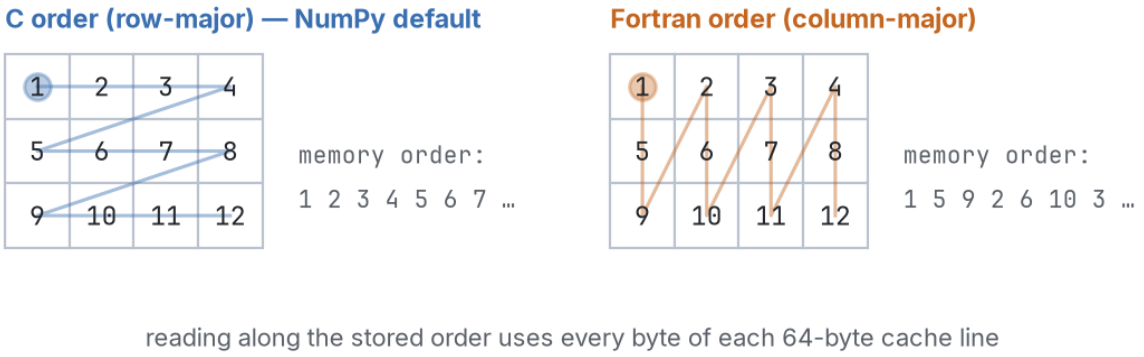


Figure 8.2 The same 3 × 4 array in both layouts, and the path memory takes through it.

FROM THE HISTORY OF DATA SCIENCE

Why two orders exist

The split goes back to the 1950s. **Fortran**, developed at IBM by John Backus's team and released in 1957, was the first widely used high-level language, built for scientists and engineers, and it stored arrays column by column. **C**, created at Bell Labs in the early 1970s, chose row by row. Both choices are perfectly sensible; they just disagree. Because NumPy's heaviest linear algebra still runs on battle-tested Fortran libraries such as LAPACK, NumPy has to speak both dialects, and it's why `np.asfortranarray` exists at all.

Why the order matters: cache locality

CPUs don't fetch single numbers from memory; they fetch **cache lines** of 64 bytes at a time. When you read data that sits contiguously, every fetch delivers eight useful `float64`s. When you stride across memory, most of each fetch is thrown away.

On a 4000 × 4000 matrix, the notebook times `sum(axis=0)` and `sum(axis=1)` on a C-ordered and on a Fortran-ordered copy. The pattern is consistent: **each layout is faster along the axis it stores contiguously.**

> **NOTE**
> Grace Hopper, a pioneer of programming languages, used to hand audiences a piece of wire about 30 cm (11.8 inches) long: the distance light travels in one nanosecond. Her point was that distance costs time, even inside a computer. Memory far from the processor is slow to reach, which is exactly why caches exist, and why reading memory in order matters so much.

> **TIP**
> You rarely choose the order on purpose, but you inherit it. `arr.T`, some SciPy routines, and data from column-oriented sources can all hand you non-C-contiguous arrays. If a downstream step is unexpectedly slow, check `.flags` and consider `np.ascontiguousarray`. It pays a copy up front, so it's worth it only if you'll use the result several times; the notebook measures the break-even point.

8.2 Temporaries: the hidden allocations

Every intermediate in an expression allocates a whole new array. A chain of operations on a big array can use several times the memory you'd expect:

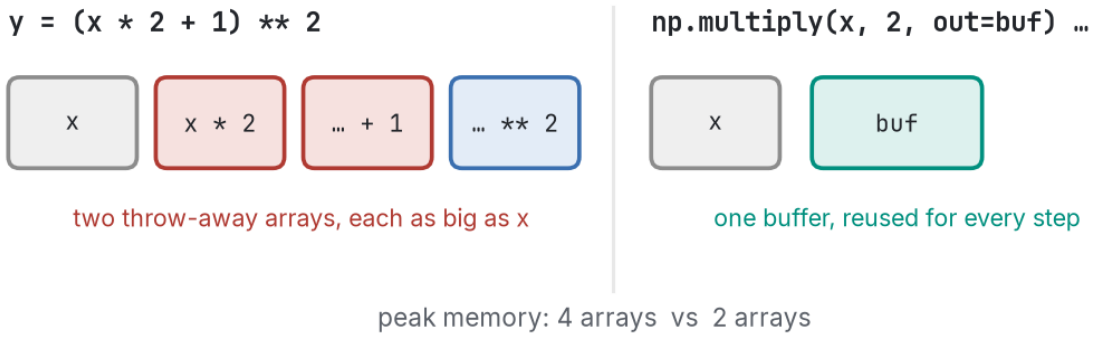


Figure 8.3 `y = (x * 2 + 1) ** 2` allocates two throw-away arrays as big as `x`. With `out=` and one reused buffer, there are none.

```
def with_temporaries(arr):
    return (arr * 2 + 1) ** 2

def in_place(arr, out):
    np.multiply(arr, 2, out=out)      # no temporaries at all
    np.add(out, 1, out=out)
    np.power(out, 2, out=out)
    return out
```

On 20 million values the in-place version is somewhat faster, but **memory is the real prize**: in-place work is what lets a job fit in RAM at all. When you own the array, the in-place operators say the same thing more readably:

```
y = x.copy()
y *= 2; y += 1; y **= 2
```

> **WATCH OUT**
> `y *= 2` mutates in place, so if `y` is a **view** of something else, you've just modified the original. Chapter 2's lesson meets the performance advice: in-place is efficient *and* dangerous, and you must know which arrays you own. `base = np.arange(10.0); view = base[2:5]; view *= 100` quietly rewrites `base`.

8.3 Measuring memory honestly

`arr.base is None` tells you whether an array owns its memory. A small profiler that adds up only the arrays that *own* their data gives the true footprint: a slice and a transpose cost nothing; a `.copy()` and an `.astype(np.float32)` each allocate a fresh buffer.

8.4 Arrays larger than RAM: `np.memmap`

A memory-mapped array lives **on disk** and is paged into memory on demand. It behaves like an ndarray but is limited by disk space, not RAM:

```
mapped = np.memmap(path, dtype=np.float64, mode="w+", shape=(10_000, 500))
for start in range(0, 10_000, 1_000):          # write it in chunks
    mapped[start:start + 1_000] = rng.random((1_000, 500))
mapped.flush()

mapped = np.memmap(path, dtype=np.float64, mode="r", shape=(10_000, 500))
mapped.mean(axis=0)                             # computed without loading it all
```

A **chunked reduction**, accumulating a running sum and count block by block, is the pattern that generalises to genuinely huge files.

8.5 Choosing the right tool

NumPy is fast when the work can be written as whole-array operations. When it can't, the answer is a different tool, not a cleverer array.

Exponential smoothing is a good example: `s[t] = α·x[t] + (1 - α)·s[t-1]`. Each step depends on the previous one, so it can't be vectorised directly, and a Python loop over 200,000 values is slow. But SciPy already implements this recurrence in C, as a digital filter:

```
from scipy.signal import lfilter

def exponential_smoothing(values, alpha):
    zi = [(1 - alpha) * values[0]]      # start exactly where the loop starts
    smoothed, _ = lfilter([alpha], [1, -(1 - alpha)], values, zi=zi)
    return smoothed
```

It matches the loop exactly and runs vastly faster. Note the `zi` argument: leave the filter's initial state at its default of zero and the two definitions disagree from the first step, leaving a transient that decays but never fully disappears. Small details matter when you swap implementations.

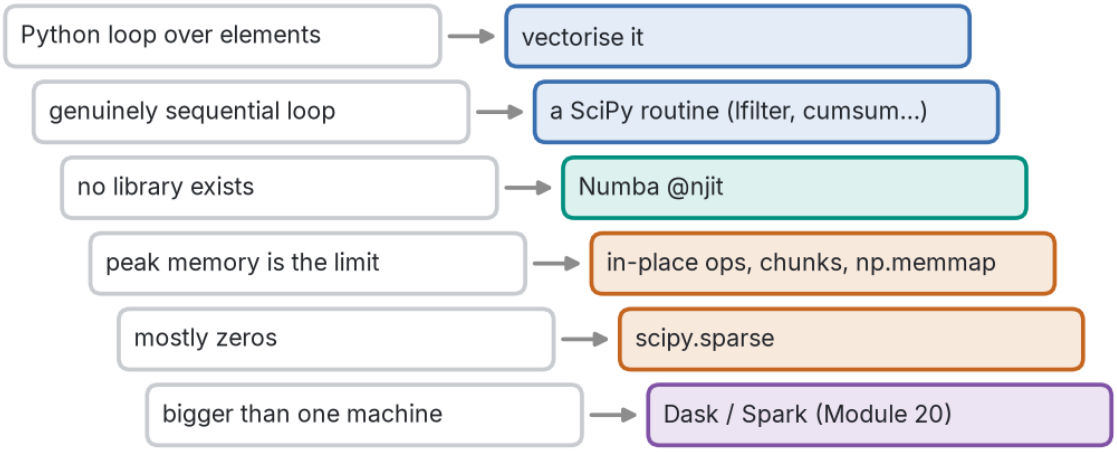


Figure 8.4 The escalation ladder: work down it before reaching for a bigger machine.

Sparse matrices

When almost every entry is zero, storing the zeros is pure waste. A 5000 × 5000 matrix with 0.1% non-zero entries takes 200 MB dense and well under 1 MB as a `scipy.sparse.csr_matrix`.

> NOTE
> One-hot encoded categories and text TF-IDF matrices are almost always more than 99% zeros. `scipy.sparse` is why a bag-of-words model over a 50,000-word vocabulary fits in memory at all (Module 18).

8.6 A performance checklist

Check	Action
Is there a Python loop over elements?	Vectorise it
Calling built-in `sum`/`max`/`min` on an array?	Use `np.sum`, `arr.max()`
Is the dtype wider than needed?	`float32` halves memory, often time too
Allocating temporaries in a hot loop?	`out=` or in-place operators
Is the array C-contiguous?	Check `.flags`; `np.ascontiguousarray` if reused
Reducing across the slow axis?	Prefer `axis=-1` on C-ordered data
Sorting just to get the top k?	`np.argpartition`
Building a huge broadcast intermediate?	`einsum`, algebra, or chunking
Is the matrix mostly zeros?	`scipy.sparse`
Does it still not fit?	`memmap`, then Dask or Spark

8.7 Try it yourself

Exercises 8.1 to 8.5 in notebook 08; solutions in notebook 10.

1. **Predict the strides** for `a = np.arange(24, dtype=np.float64).reshape(2, 3, 4)`, `a.T`, `a[::2]`, `a[:, ::2, :]` and `a.reshape(6, 4)`. Then verify.
2. **Row-major versus column-major.** Time `sum(axis=0)` and `sum(axis=1)` on a 3000 × 3000 array in both orders. Predict the fastest combination first, and explain it with cache lines.
3. **Eliminate the temporaries.** Rewrite `np.sqrt(a**2 + b**2) / (c + 1)` with `out=` and at most two buffers.
4. **Chunked statistics.** Compute the mean and standard deviation of a memmapped array **in one pass** by accumulating count, sum and sum of squares, and mind the catastrophic cancellation from Chapter 3.
5. **Know when to stop.** Implement a cumulative maximum three ways (Python loop, `np.maximum.accumulate`, chunked) on 10 million elements, and explain the ranking.

KEY TAKEAWAYS

- An ndarray is a **flat buffer plus strides**. Reshape and transpose only rewrite the strides, which is why they're free.
- **Contiguity governs cache behaviour.** Reduce along the axis the array stores contiguously; check `.flags` when something is mysteriously slow.
- Every intermediate **allocates a full array**. Use `out=` or in-place operators when memory is tight.
- **In-place operations on a view mutate the original.**
- `.base is None` tells you whether an array owns its memory.
- `np.memmap` handles arrays bigger than RAM; chunked reductions generalise the pattern.
- For a genuinely sequential loop, look for a **SciPy routine** before reaching for Numba.
- `scipy.sparse` for matrices that are mostly zeros.

CHAPTER 09

09

Project: Two Algorithms From Scratch

Linear regression and k-means, in nothing but NumPy

Notebook 09_module_project.ipynb · About 120 minutes

IN THIS CHAPTER

- Implementing **linear regression** three ways (SVD, normal equations, gradient descent) and adding Ridge.
- Seeing **multicollinearity** happen, and watching Ridge tame it.
- Implementing **k-means** with k-means++ initialisation and independent restarts.
- Following the **scikit-learn estimator contract**, and checking every result against scikit-learn itself.
- Why being a few times slower than scikit-learn is a *good* result.

There's a particular confidence that comes from having built something yourself. When `LinearRegression().fit()` returns a suspicious coefficient in Module 11, or `KMeans` gives different clusters on every run, you'll know why, because you will have written the loop. This project builds two of the most-used algorithms in data science with nothing but NumPy, and only brings in scikit-learn at the end, as the examiner.

Chapter	What the project uses from it
1	Vectorisation instead of Python loops
2	Shapes, views versus copies, `reshape(-1, 1)`
3	`float64` for accumulation, condition numbers
4	Broadcasting for pairwise distances
5	`argmin`, boolean masks, `keepdims`
6	`lstsq`, the normal equations, the SVD
7	Seeded generators, reproducible initialisation, `SeedSequence.spawn`
8	Avoiding temporaries; choosing the cheap operation

9.1 Part 1: linear regression

The problem

Given a design matrix `X` of shape `(n, p)` and targets `y` of shape `(n,)`, find the coefficients β that minimise the sum of squared residuals, $\|X\beta - y\|^2$.

The estimator

`LinearRegressionScratch` follows the scikit-learn contract from Module 01: the constructor only stores settings, `fit` learns `coef_` and `intercept_` (trailing underscores) and returns `self`, and `predict` and `score` use what was learned. It's written as a `@dataclass`, with the fitted attributes marked `field(init=False)` so they stay out of the constructor.

```
@dataclass
class LinearRegressionScratch:
    fit_intercept: bool = True
    alpha: float = 0.0                # 0 → ordinary least squares; > 0 → Ridge
    solver: str = "lstsq"             # "lstsq", "normal" or "gradient"
    learning_rate: float = 0.01
    n_iterations: int = 1000

    coef_: np.ndarray = field(init=False, default=None)
    intercept_: float = field(init=False, default=0.0)
```

A `_validate` step coerces inputs to `float64` (Chapter 3 showed what `float32` does to accumulation), turns a 1-D `X` into a column, and refuses mismatched row counts with a clear message.

Three solvers

`lstsq`, **via the SVD.** Robust, and it survives rank deficiency. Ridge is added in an elegant way: stack `√α · I` underneath `X` and zeros underneath `y`, then solve ordinary least squares on the taller system. That's mathematically identical to Ridge and more stable than forming $X^TX + \alpha I$:

```
penalty = np.sqrt(self.alpha) * np.eye(n_features)
if self.fit_intercept:
    penalty[0, 0] = 0.0               # never penalise the intercept
Xd = np.vstack([Xd, penalty])
y = np.concatenate([y, np.zeros(n_features)])
beta, *_ = np.linalg.lstsq(Xd, y, rcond=None)
```

`normal`, **the normal equations**, solved with `np.linalg.solve`, never with `inv` (Chapter 6).

`gradient`, **batch gradient descent**: start from zeros and repeatedly step downhill along the gradient of the mean squared error. It's how neural networks are trained (Module 17), so it's worth writing once by hand:

```
for _ in range(self.n_iterations):
    residuals = Xd @ beta - y
    gradient = (2 / n_samples) * (Xd.T @ residuals)
    beta -= self.learning_rate * gradient
```

NOTE

Following the slope downhill is an old idea: the French mathematician Augustin-Louis Cauchy described the method of steepest descent in 1847, for solving systems of equations from astronomy. A century and a half later it trains every deep-learning model on the planet.

Testing it

On 500 simulated samples with known coefficients (intercept 10; slopes 3.0, −1.5, 0.7 and 2.2), all three solvers recover the truth, and the `lstsq` version matches scikit-learn's `LinearRegression` coefficients and R^2 exactly. Ridge with `alpha=5` matches `sklearn.linear_model.Ridge` too. Gradient descent's loss curve falls steadily onto the closed-form answer.

Where the solvers differ: multicollinearity

On well-conditioned data the solvers agree. The differences appear exactly where Chapter 6 said they would. The notebook builds a design in which column 2 is column 0 plus a whisper of noise (standard deviation one millionth), and fits both solvers.

Both fit the data equally well, with almost identical R^2. But the coefficients are wild: the model can't tell columns 0 and 2 apart, so it assigns them enormous, offsetting weights. **This is what multicollinearity looks like**, and it's why the coefficients of an unregularised model on correlated features must never be interpreted.

Ridge fixes it. With `alpha=1.0` the coefficient vector's length becomes small and sensible, and R^2 is barely any worse.

FROM THE HISTORY OF DATA SCIENCE

Why it's called "regression"

The word comes from Francis Galton, who in 1886 studied the heights of parents and their adult children. Very tall parents tended to have children who were tall, but *less* extremely so; very short parents had children who were short, but closer to average. Galton called the effect "regression towards mediocrity", meaning towards the mean, and the line he drew through his data gave the whole method its name. Today we'd call that effect regression to the mean, and it's still one of the most common traps in interpreting data: an extreme measurement is usually followed by a less extreme one, with no cause required.

9.2 Part 2: k-means clustering

The algorithm

1. Choose `k` initial centroids.
2. **Assign** each point to its nearest centroid.
3. **Update** each centroid to the mean of its assigned points.
4. Repeat steps 2 and 3 until the centroids stop moving.

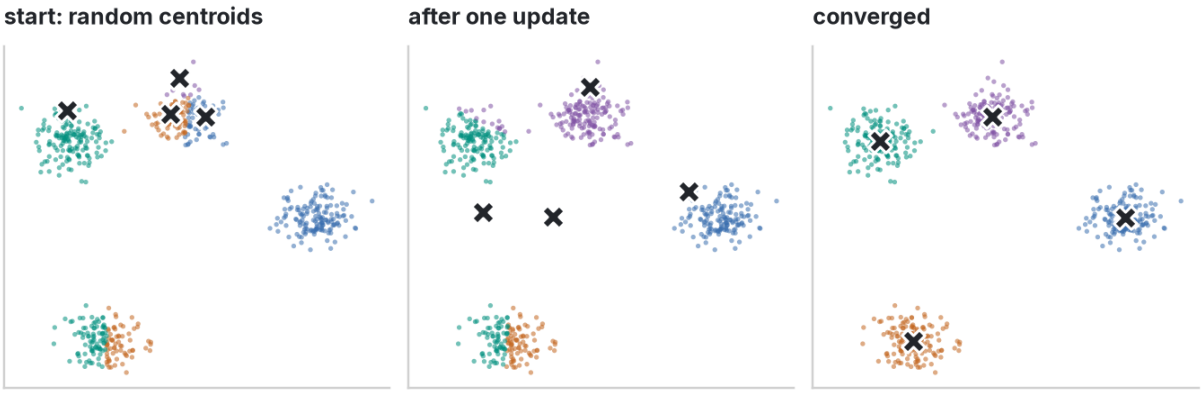


Figure 9.1 Three snapshots of k-means on four blobs: a poor random start, the picture after a single update, and the converged solution. The X marks are the centroids.

Step 2 is Chapter 4's broadcasting exercise; step 3 is a masked mean per cluster (Chapter 5).

Distances without a giant tensor

The naive way to compute every point-to-centroid distance builds an `(n, k, p)` difference tensor. For 100,000 points, 10 clusters and 50 features, that tensor alone is 400 MB. Expanding the square avoids it entirely:

$$\|x - c\|^2 = \|x\|^2 - 2\, x \cdot c + \|c\|^2$$

```
@staticmethod
def _squared_distances(X, centroids):
    x_norm = (X**2).sum(axis=1, keepdims=True)    # (n, 1)
    c_norm = (centroids**2).sum(axis=1)            # (k,)
    cross = X @ centroids.T                        # (n, k)
    distances = x_norm - 2 * cross + c_norm        # broadcasts to (n, k)
    return np.maximum(distances, 0.0)              # clip tiny negatives
```

The result needs just 8 MB. Broadcasting, `keepdims` and a matrix product, all in five lines. The final `np.maximum` is Chapter 3 in action: cancellation in the subtraction can leave tiny negative numbers where the true answer is zero, so we clip them.

k-means++ initialisation

Where the centroids start matters enormously. **k-means++** picks the first centroid at random, then each next one with probability proportional to its squared distance from the nearest centroid already chosen. That spreads the seeds out and sharply reduces the chance of a bad local optimum.

```
closest = self._squared_distances(X, centroids[:i]).min(axis=1)
centroids[i] = X[rng.choice(n_samples, p=closest / closest.sum())]
```

An **empty cluster** (a centroid nobody chose) is re-seeded at the worst-fitted point, which is what scikit-learn does too.

Why `n_init` is not optional

k-means converges to a **local** optimum that depends entirely on where it started. Run it once and you're gambling. The notebook runs 30 single-restart fits and finds a wide spread between the best and worst final inertia. So `fit` runs `n_init` independent restarts, each with its own generator spawned from one parent seed (Chapter 7), and keeps the best:

```
parent = np.random.SeedSequence(self.random_state)
generators = [np.random.default_rng(s) for s in parent.spawn(self.n_init)]
```

Testing it

On 1,500 points in four blobs, our inertia matches scikit-learn's `KMeans` to within a tiny relative gap. An elbow plot of inertia against `k` bends clearly at `k = 4`.

> NOTE
>
> Comparing `labels_` element by element would be wrong: cluster 0 in our run may be cluster 2 in scikit-learn's. The **Adjusted Rand Index** measures whether the same points ended up *together*, which is the only thing that matters. Ours agrees with scikit-learn's partition almost perfectly.

> FROM THE HISTORY OF DATA SCIENCE
>
> #### A clustering algorithm that waited 25 years to be published
>
> The procedure in this chapter is often called **Lloyd's algorithm**. Stuart Lloyd devised it at Bell Labs in 1957, as a way of quantising signals for pulse-code modulation, but it circulated only as an internal memo and wasn't formally published until 1982. Meanwhile James MacQueen coined the name "k-means" in 1967. The k-means++ seeding you just implemented is much younger: David Arthur and Sergei Vassilvitskii published it in 2007, and scikit-learn uses it by default.

Performance: ours versus scikit-learn

On 50,000 points with 10 features and `k = 8`, our version lands within a fraction of a percent of scikit-learn's inertia, and is a few times slower. That's a **good** result for pure NumPy. The gap comes from Cython inner loops, Elkan's algorithm (which uses the triangle inequality to skip distance computations that can't change an assignment) and multithreading, not from anything wrong with our approach.

The lesson is Chapter 1's: **use the library in production, but know what it's doing.**

9.3 Verification

The notebook ends with a checklist, every item of which must pass: our linear regression and Ridge match scikit-learn; gradient descent converges to the closed-form answer; predictions have the right shape; R^2 lies between 0 and 1; our k-means partition matches scikit-learn's (ARI above 0.99) with inertia within 1%; every point is assigned; no cluster is empty; and no squared distance is negative.

9.4 Your turn: extensions

#	Extension	Difficulty
1	**Promote** both classes into `src/dstoolkit/models.py`, with tests for the shape guards and the scikit-learn agreement.	★
2	**Weighted least squares:** $\beta = (X^\top W X)^{-1} X^\top W y$, *without* forming the `(n, n)` matrix `W` (80 GB for 100,000 samples).	★★
3	**Mini-batch k-means:** update centroids from random subsets; compare speed and final inertia on 500,000 points.	★★
4	**Silhouette score** in vectorised NumPy, used to choose `k` instead of the elbow. Watch the `(n, n)` memory.	★★★
5	**Polynomial features** with a `degree` parameter. Show the condition number exploding, and Ridge rescuing degree 10.	★★
6	**Elkan's algorithm:** skip impossible distance computations; measure how much of the gap to scikit-learn it closes.	★★★

9.5 What you built

Two production-shaped estimators that follow the scikit-learn contract and drop into a `Pipeline`; validate their inputs and fail with useful messages; use `lstsq` rather than `inv` and stay correct on rank-deficient data; compute distances by expansion rather than a difference tensor; restart from independent, seeded streams and keep the best; and agree with scikit-learn to within floating-point tolerance.

The next module is pandas. Everything you've learned about shapes, dtypes, views and vectorisation carries straight over: a DataFrame is a collection of labelled NumPy arrays with a great deal of convenience on top.

KEY TAKEAWAYS

- `lstsq` **is the default solver**: robust, and it survives collinearity. Normal equations via `solve`, never `inv`.
- Ridge can be written as ordinary least squares on an **augmented** system.
- Gradient descent reaches the same answer by walking downhill; it's how neural networks learn.
- **Collinear features make coefficients meaningless** even when the fit looks perfect. Regularise before you interpret.
- k-means is **assign, update, repeat**; expand $\|x - c\|^2$ to avoid a giant tensor.
- k-means finds a **local** optimum: use **k-means++** and **several restarts**.
- Compare clusterings with the **Adjusted Rand Index**, never label by label.
- Use the library in production, but know what it's doing.

Before You Move On

You're ready for Module 03 when you can answer every question below *without looking anything up*. If one makes you hesitate, the chapter in brackets is where to go back to.

Self-assessment

1. Why is a Python list of numbers both slower *and* larger than an ndarray? *(Chapter 1)*
2. Why does `arr[1:3]` behave differently from `lst[1:3]`? *(Chapter 2)*
3. What does `axis=0` actually collapse? *(Chapter 5)*
4. When does `keepdims=True` matter, and what breaks without it? *(Chapter 5)*
5. What are the two broadcasting rules? *(Chapter 4)*
6. Why does `int32_arr.sum()` not overflow while `int32_arr * 400` does? *(Chapter 3)*
7. Why should you never call `np.linalg.inv` to solve a system? *(Chapter 6)*
8. What does a condition number of 10^{16} tell you? *(Chapter 6)*
9. Why is `np.random.seed` a poor choice for reproducibility? *(Chapter 7)*
10. What are strides, and why is `arr.T` free? *(Chapter 8)*

The three traps this module exists to prevent

If you remember nothing else, remember these. Each produces **wrong numbers rather than an exception**.

1. **A slice is a view.** `arr[1:3] = 0` writes into the original array. Lists copy on slice; arrays don't. *(Chapter 2)*
2. **Integers wrap silently.** `int32` arithmetic past 2,147,483,647 doesn't raise; it goes negative. *(Chapter 3)*
3. **`(n,)` and `(n, 1)` broadcast to `(n, n)`.** No error, an answer of the wrong shape, and a mean over the wrong thing. *(Chapter 4)*

Where the exercises and solutions live

Each of notebooks 01–08 ends with five exercises, 40 in all, indexed in `modules/02-numpy-numerical-computing/exercises/README.md`. The worked solutions are in notebook `10_exercise_solutions.ipynb`, and the project adds six open-ended extensions.

> WATCH OUT
> **Do the exercises before reading the solutions.** Reading a solution feels like learning, and isn't.

Further reading

Resource	Why it's worth your time
NumPy: the absolute basics for beginners	The official tour: a fast second pass over Chapters 1–2.
"Array programming with NumPy" (Harris et al., *Nature*, 2020)	The paper by the maintainers: the clearest account of *why* the design is what it is.
NumPy broadcasting docs	The rules stated formally, with a shape table worth memorising.
"What Every Computer Scientist Should Know About Floating-Point Arithmetic" (Goldberg, 1991)	The reference behind Chapter 3. Dense, and worth it.
NumPy random sampling guide	Why `Generator` replaced `RandomState`, and how `SeedSequence` works.
Introduction to Linear Algebra (Gilbert Strang)	If Chapter 6 felt thin, this is the book to read alongside Modules 11–19.

Sources for the Stories

The history boxes in this book simplify, as stories must, but none of them is invented. These are the sources behind each one, in the order they appear, so that you can read the full versions and check the details for yourself.

Chapter 1

- **APL.** K. E. Iverson, *A Programming Language*, Wiley, 1962; K. E. Iverson, "Notation as a Tool of Thought", 1979 ACM Turing Award Lecture, *Communications of the ACM* 23(8), 1980.
- **Numeric, Numarray and NumPy; LIGO and the black-hole image.** C. R. Harris et al., "Array programming with NumPy", *Nature* 585, 2020; T. E. Oliphant, *A Guide to NumPy*, Trelgol Publishing, 2006.

Chapter 2

- **"Matrix".** J. J. Sylvester, "Additions to the articles 'On a new class of theorems' and 'On Pascal's theorem'", *Philosophical Magazine* 37, 1850; A. Cayley, "A Memoir on the Theory of Matrices", *Philosophical Transactions of the Royal Society of London* 148, 1858.

Chapter 3

- **Ariane 5 Flight 501.** J.-L. Lions et al., *ARIANE 5 Flight 501 Failure: Report by the Inquiry Board*, European Space Agency and CNES, 19 July 1996.
- **The Patriot missile at Dhahran.** US General Accounting Office, *Patriot Missile Defense: Software Problem Led to System Failure at Dhahran, Saudi Arabia*, GAO/IMTEC-92-26, February 1992.
- **IEEE 754.** *IEEE Standard for Binary Floating-Point Arithmetic*, ANSI/IEEE Std 754-1985; ACM A. M. Turing Award citation for W. Kahan, 1989.

Chapter 4

- **Nearest neighbours.** E. Fix and J. L. Hodges, *Discriminatory Analysis. Nonparametric Discrimination: Consistency Properties*, Technical Report 4, USAF School of Aviation Medicine, Randolph Field, Texas, 1951.

Chapter 5

- **Florence Nightingale.** F. Nightingale, *Notes on Matters Affecting the Health, Efficiency, and Hospital Administration of the British Army*, London, 1858; Royal Statistical Society, membership records.
- **Box plots and fences.** J. W. Tukey, *Exploratory Data Analysis*, Addison-Wesley, 1977.

Chapter 6

- **The condition number.** A. M. Turing, "Rounding-off errors in matrix processes", *Quarterly Journal of Mechanics and Applied Mathematics* 1(1), 1948.
- **Ceres and least squares.** C. F. Gauss, *Theoria Motus Corporum Coelestium*, 1809; A.-M. Legendre, *Nouvelles méthodes pour la détermination des orbites des comètes*, 1805; for the history, S. M. Stigler, *The History of Statistics*, Harvard University Press, 1986.
- **PCA.** K. Pearson, "On Lines and Planes of Closest Fit to Systems of Points in Space", *Philosophical Magazine* 2(11), 1901; H. Hotelling, "Analysis of a complex of statistical variables into principal components", *Journal of Educational Psychology* 24, 1933.
- **The Netflix Prize.** J. Bennett and S. Lanning, "The Netflix Prize", *Proceedings of KDD Cup and Workshop*, 2007; Y. Koren, R. Bell and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems", *IEEE Computer* 42(8), 2009.

Chapter 7

- **RAND's random digits.** RAND Corporation, *A Million Random Digits with 100,000 Normal Deviates*, Free Press, 1955.
- **Monte Carlo.** N. Metropolis and S. Ulam, "The Monte Carlo Method", *Journal of the American Statistical Association* 44(247), 1949; N. Metropolis, "The Beginning of the Monte Carlo Method", *Los Alamos Science*, Special Issue, 1987.
- **Buffon's needle.** G.-L. Leclerc, Comte de Buffon, *Essai d'arithmétique morale*, 1777.
- **The bootstrap.** B. Efron, "Bootstrap Methods: Another Look at the Jackknife", *The Annals of Statistics* 7(1), 1979; International Prize in Statistics, 2018.

Chapter 8

- **Fortran.** J. W. Backus et al., "The FORTRAN automatic coding system", *Proceedings of the Western Joint Computer Conference*, 1957.
- **Grace Hopper's nanosecond.** Smithsonian National Museum of American History, collection of Grace Hopper's "nanoseconds".

Chapter 9

- **Gradient descent.** A.-L. Cauchy, “Méthode générale pour la résolution des systèmes d’équations simultanées”, *Comptes rendus de l’Académie des sciences* 25, 1847.
- **Regression to the mean.** F. Galton, “Regression Towards Mediocrity in Hereditary Stature”, *Journal of the Anthropological Institute of Great Britain and Ireland* 15, 1886.
- **k-means.** S. P. Lloyd, “Least squares quantization in PCM”, *IEEE Transactions on Information Theory* 28(2), 1982 (Bell Labs memorandum, 1957); J. MacQueen, “Some methods for classification and analysis of multivariate observations”, *Proceedings of the Fifth Berkeley Symposium*, 1967; D. Arthur and S. Vassilvitskii, “k-means++: The Advantages of Careful Seeding”, *SODA*, 2007.